Gymnasium Oberwil

Maturarbeit

Bildsynthese von menschlichen Gesichtern mithilfe von Kuenstlicher Intelligenz

Eine Arbeit ueber Kuenstliche Neuronale Netze, Convolutional Neural Networks, Autoencoders, Generative Modelle und Tensorflow/Keras

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019
LaTeX

Inhaltsverzeichnis

Inhaltsverzeichniss **I**

Vorwort **III**

Einleitung **IV**

Konvention und Notation **V**

1 Machinelles Lernen **1**
1.1 Allgemeine Begriffe . . . 2
1.1.1 Daten . . . 2
1.1.2 Modelle . . . 2
1.2 Training . . . 3
1.2.1 Kostenfunktionen . . . 3
1.2.2 Gradientenverfahren . . . 4
1.2.3 Stochastisches Gradientenverfahren fuer Machine Learning . . . 6
1.2.4 Adam Optimizer . . . 7
1.3 Trainingsphaenomene . . . 7
1.3.1 Overfitting . . . 7

2 Deep Learning und Künstliche Neuronale Netze **8**
2.1 Perzeptron . . . 8
2.1.1 Was kann ein Perzeptron erlernen? . . . 9
2.1.2 Biologische Analogie . . . 9
2.2 Erweiterung der Kuenstlichen Neuronen . . . 10
2.2.1 Kuenstliche Neuronen im Allgemeinen . . . 10
2.2.2 Perzeptronen als Kuenstliche Neuronen . . . 10
2.2.3 Stueckweise lineare Neuronen . . . 10
2.2.4 Sigmoide Neuronen . . . 11
2.3 Topologie der Kuenstlichen Neuronalen Netzen . . . 11
2.4 Vorwaertspropagierung . . . 12
2.4.1 Modellparameter initalisieren . . . 15
2.5 Rueckwaertspropagierung . . . 16
2.6 Universal Approximation Theorem . . . 20
2.7 Vanishing Gradient Problem . . . 20
2.7.1 Kreuzentropie-Kostenfunktion . . . 20
2.7.2 ReLU . . . 20

3 Convolutional Neural Networks **21**
3.1 Bilder als Tensoren . . . 21
3.2 Topologie von CNNs . . . 22

3.3 Filter . . . 23
3.3.1 Filter in der Bildverarbeitung . . . 23
3.3.2 Filter in CNNs . . . 24
3.3.3 Filteroperation intutitiv . . . 24
3.3.4 Filteroperationen als diskrete Faltungen . . . 25
3.3.5 Padding . . . 26
3.3.6 Stride . . . 27
3.3.7 Mehrere Filter . . . 27
3.3.8 Wieso Filter? . . . 27
3.4 Convolutional Layer . . . 28
3.5 Pooling Layer . . . 28
3.6 Fully-connected Layer . . . 29

4 Autoencoder 30
4.1 Topologie von Autoencodern . . . 30
4.2 Mathematische Betrachtung . . . 30
4.3 Bedeutung von Autoencodern . . . 31
4.3.1 Datenkompression . . . 31
4.3.2 Datengenerierung . . . 31
4.3.3 Eigenschaften erkennen . . . 31
4.4 Variational Autoencoder? . . . 31
4.5 Detangled Autoencoder? . . . 31
4.6 Autoencoder als generative Modelle . . . 31
4.7 Programmiertools . . . 31
4.7.1 Tensorflow . . . 31

5 Entwicklung eines Autoencoders 32
5.1 Planung/Methoden . . . 32
5.1.1 Idee . . . 32
5.1.2 Methoden/Tools . . . 32
5.1.3 Trainingsdaten . . . 32
5.2 Daten-Normalisierung . . . 32
5.2.1 Daten laden . . . 32
5.2.2 Facealigner . . . 32
5.2.3 Zuschneiden . . . 33
5.3 Programmieren . . . 33

A Beispielsanhang 34

Literatur 35

Abbildungsverzeichniss 36

Tabellenverzeichniss 37

Selbstständigkeitserklärung 38

Vorwort

Einleitung

Die Leitfrage lautet: **Wie können mithilfe von Machine Learning Bilder von künstlichen meschlichen Gesichtern generiert werden?**

Konvention und Notation

Beschriftung

Zahlen und Tensoren

a	ein Skalar (Zahl)
$\boldsymbol{a}$	ein Vektor
$\boldsymbol{a} \in \mathbb{R}^n$	Vektor mit n Komponenten
$\boldsymbol{A}$	eine Matrix
$\boldsymbol{A} \in \mathbb{R}^{n \times m}$	Matrix mit n Zeilen und m Spalten
$\overline{\mathbf{A}}$	ein Tensor
$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \cdots$	a
$\mathfrak{a}, \mathfrak{b}, \mathfrak{c}, \mathfrak{d}, \mathfrak{e}, \mathfrak{f}, \mathfrak{g}, \cdots$	a

Mengen

(a, b)	ein geordnetes Paar (2-Tupel) der Elemente a und b
$\mathbb{A}$	eine Menge
$\mathbb{R}$	die Menge aller Reeller Zahlen
$\mathbb{Z}$	die Menge aller ganzer Zahlen
$\mathbb{N}$	die Menge aller natuerlicher Zahlen
$\mathbb{E}$	die Menge aller gerader (engl.: even) Zahlen
$\mathbb{O}$	die Menge aller ungerader (engl.: odd) Zahlen
$\{a, b\}$	die Menge, welche aus Elementen a und b besteht
$\{1, \ldots, n\}$	Menge aller ganzen Zahlen von 1 bis n
$\in$	... ist Element von ...

Indexierung

a_i	i-te Vektorkomponente mit Startindex 1
$(\boldsymbol{A})_{i,j} =$	Matrixelement in der i-ten Zeile und der j-ten Spalte
$A_{i,j}$	Matrixelement in der i-ten Zeile und der j-ten Spalte
$\boldsymbol{A}_{i,:}$	Zeile i der Matrix $\boldsymbol{A}$
$\boldsymbol{A}_{:,i}$	Spalte i der Matrix $\boldsymbol{A}$

Lineare Algebra Operationen

$\boldsymbol{v} \cdot \boldsymbol{w}$	Skalarprodukt von $\boldsymbol{v}$ mit $\boldsymbol{w}$
$\boldsymbol{A}^\top$	das Transponierte einer Matrix $\boldsymbol{A}$
$\boldsymbol{A} \odot \boldsymbol{B}$	Elementeweises (Hadamard) Produkt
$\boldsymbol{A} * \boldsymbol{B}$	diskrete Faltung von $\boldsymbol{A}$ ueber $\boldsymbol{B}$

Infinitesimalrechnung/Differentialrechnung

$f'(x)$	Ableitung der Funktion f bezueglich seinem Argument x
$\frac{\mathrm{d}y}{\mathrm{d}x}$	Ableitung von y bezueglich x
$\frac{\partial y}{\partial x}$	Partielle Ableitung von y bezueglich x
$\boldsymbol{\nabla} y$	Gradient von y
$\boldsymbol{\nabla}_{\boldsymbol{x}} y$	Gradient von y bezueglich $\boldsymbol{x}$ (Vektor)

Funktionen

$f : \mathbb{A} \to \mathbb{B}$	Funktion f mit Definitionsmenge $\mathbb{A}$ und Zielmenge $\mathbb{B}$
$f(x)$	Funktion f mit Argument x (Skalar)
$f(\boldsymbol{v})$	Funktion f mit Argument $\boldsymbol{v}$ (Vektor)
$\boldsymbol{f}[\boldsymbol{v}]$	Vektorisierte Funktion $\boldsymbol{f}$ mit Argument $\boldsymbol{v}$ (Vektor)
$\mathcal{A}$	ein Funktionenraum
$f * g$	Faltung von f ueber g

Statistik

$\mathcal{N}$	Gauss'sche Normalverteilung
ϕ	Gauss'sche Dichtefunktion
μ	Erwartungswert/Mittelwert
σ^2	Varianz

Sonstiges

$\forall$	"fuer jedes ..."
$\therefore$	"... daraus folgt, dass ..."
$\exists$	"... existiert"

Kapitel 1

Machinelles Lernen

Der Themenbereich des **Machinellen Lernens** beschäftigt sich mit Algorithmen und mathematischen Modellen, welche von selber lernen, Probleme zu loesen. Hierbei wird nicht explizit einprogrammiert, wie das Modell das Problem zu loesen hat, statdessen wird das Modell trainiert, optimiert sich von selbst und findet selber einen Weg, das Problem zu loesen. Die Grundidee dabei ist, dass man Daten erfasst, generiert oder misst, welche analysiert werden sollen. Innerhalb diesen Daten existieren gewisse Gesetzmaessigkeiten und Mustern. Diese Muster sollen vom Modell erkannt werden und verallgemeinert werden. Nach dem erfolgreichen Lernen, kann das Modell **Vorhersagen** zu neuen Daten machen.

Man unterscheiden zwischen zwei Arten von Maschinellen Lernen:

- **Ueberwachtes Lernen** (engl.: supervised learning) ist ein Lernverfahren, bei welchem die Daten aus zwei Teilen bestehen, aus Inputs und Outputs. Man bezeichnet dabei die Outputs als Labels. Die Aufgabe des Modells ist es eine **Korrelation** zwischen den Inputs und den Labels zu erlernen und so ihre Beziehung zueinander zu verstehen. Anhand der Informationen, welche in den Inputs enthalten sind, sind dann die Labels vorherzusagen. Man gleicht dann die vorhergesagten Labels des Modells mit den wahren Labels ab (man ueberwacht sie) und bewertet so ihre Faehigkeiten.

  Voraussetztung dafuer ist, dass die Daten eben "gelabelt" sein muessen. Man muss schon im vorhinein Daten haben, bei welchen man die gewuenschten Labels hat. Natuerlich muss auch die erwaehnte Korrelation bestehen. Falls kein Zusammenhang zwischen den Inputs und den Outputs besteht, kann das Modell auch keine Vorhersagen machen und so auch nichts erlernen.

- **Unueberwachtes Lernen** (engl.: unsupervised learning) ist ein anderes Lernverfahren, bei welchem eben diese Labels nicht vorhanden sind. Dem Modell stehen nur die Inputdaten zur Verfuegung. Diese werden erneut analyisert und das Modell soll Muster herausextrahieren, welche sich von sonstigem zuefaelligen Rauschen (durch z.B. Messungenauigkeiten erzeugt) unterscheiden.

Der Grossteil der Modelle des Maschinellen Lernens benutzten ueberwachtes Lernen, da es auch deutlich mehr Anwendungsmoeglichkeiten besitzt. Es gibt nur wenige Arten von unueberwachtem Lernen, darunter sind: Hauptkomponentenanalyse und Generative Adversial Networks. Aus diesem Grund, werden wir im folgenden uns grundsaetzlich mit ueberwachtem Lernen beschaeftigen.
Im Verlauf dieser Arbeit, werden auch noch Autoencoder betrachten, welche eine Art Mischform darstellen. Man spricht von semiueberwachtem Lernen.

Um die neuen Konzepte in den folgenden Abschnitten besser zu verstehen, werden wir ein bestimmtes Beispielmodell betrachten:

Das Ziel ist den Preis eines Occasion Autos vorherzusagen, anhand der Anzahl gefahrener Kilometer und anhand des Alters des Autos.

(Mehrere, *Supervised learning — Wikipedia, The Free Encyclopedia*)
(Mehrere, *Unsupervised learning — Wikipedia, The Free Encyclopedia*)

1.1 Allgemeine Begriffe

1.1.1 Daten

Um ein Modell zu trainieren braucht man Daten. Diese Daten bestehen immer aus **Inputs** und aus **Outputs**. Man unterscheidet hierbei zwei Arten von Outputs. Die **Labels** sind die erwarteten Outputs, welche die gewueschten Zielwerte sind. Die **Vorhersagen** sind die Outputs, welche das Modell produziert und hoffentlich moeglichst genau mit den Labels uebereinstimmen.
Diese Daten fuer Training kommen in der Form eines **Trainingsdatensatz** $\mathbb{X}$. Dabei handelt es sich um eine Menge von **Samples**, welche jeweils aus Inputs und Labels bestehen. Die Inputs werden in einem Vektor

$$\boldsymbol{x} = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}^\mathsf{T}$$

und die Labels in einem Vektor

$$\hat{\boldsymbol{y}} = \begin{pmatrix} \hat{y}_1 & \hat{y}_2 & \cdots & \hat{y}_m \end{pmatrix}^\mathsf{T}$$

zusammengefasst. Auch die Vorhersagen werden in einem Vektor

$$\boldsymbol{y} = \begin{pmatrix} y_1 & y_2 & \cdots & y_m \end{pmatrix}^\mathsf{T}$$

zusammengefasst. Somit ist ein Trainingssample nichts anderes als ein Paar $(\boldsymbol{x}_i, \hat{\boldsymbol{y}}_i)$ eines Inputsvektor $\boldsymbol{x}$ und eines Labelvektor $\hat{\boldsymbol{y}}$.

Die Inputs bestehen aus gewissen **Attributen**, welche die Merkmals*typen* der Daten sind. Die sogennanten **Features** sind dann die spezifischen Werte der Attribute eines Samples. Der Algorithmus soll anhand dieser Attribute seine Vorhersagen machen. Diese Vorhersagen werden dann mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (keine Ueberanpassung) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs anhand von Mustern und Gesetzmassigkeiten.

Um eine endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz $\mathbb{T}$, welcher nicht Teil des Trainingsdatensatz ist, genutzt um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

Um die soeben eingefuerten Begriffe zu verdeutlichen, betrachten wir unser Autobeispiel. Die Inputdaten sind die Anzahl gefahrener Kilometer und die Alter von verschieden Autos. Sie stellen

1.1.2 Modelle

Ein **Modell** ist eine mathematische Funktion $f\colon \mathbb{R}^n \to \mathbb{R}^m$, welche die Inputs auf die Outputs abbildet $\boldsymbol{y} = f(\boldsymbol{x})$. Man kann diese Funktion gewissermassen als die Hypothese der Beziehung zwischen den Inputs und den Labels betrachten, welche das Modell aufgestellt hat. Im Zusammenhang mit Wertevorhersagen spricht man von einem Regressionsmodell. Das Verhalten

dieses Modells wird bestimmt durch seine **Modellparameter** $\lambda_1, \lambda_2, \ldots, \lambda_n$. Sie bestimmen, wie die Hypothese des Modells lautet. Das Ziel ist es, die Modellparameter so einzustellen, dass die Vorhersagen $\boldsymbol{y}$ besser mit den Labels $\hat{\boldsymbol{y}}$ uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Parametern vorgenommen werden, bis das Modell die gewuenschten Resultate liefert.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang der Form $y = \lambda_1 x + \lambda_0$ zwischen den Features und den Labels besteht. Nun muessten nur noch λ_0 und λ_1 bestimmt werden.
Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen.

Fuer Machine Learning haben sich gewisse Modelle besonders guet etabliert, darunter waeren: Support Vector Machines, Evolutionäre Algorithmen, und Künstliche Neuronale Netze. Diese Arbeit wird sich vorwiegend mit Neuronalen Netzen auseinandersetzen.

BEISPIELMODELL GLEICHES WIE VORHIN

Abbildung 1.1: Schema eines Modells

Abbildung 1.2: beschriebenes Beispielmodell

1.2 Training

1.2.1 Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Modells bewertet werden. Dafuer gibt es sogenannte **Kostenfunktionen** (auch Verlust- oder Fehlerfunktionen). Sie sollen ein Mass fuer die Abweichung des Outputs $\boldsymbol{y}$ vom Label $\hat{\boldsymbol{y}}$ sein.

Die Fehler der einzelnen Outputs $c(y_i, \hat{y}_i)$ werden aufsummiert um den Fehler $C(\boldsymbol{y}_j, \hat{\boldsymbol{y}}_j)$ (wobei $\boldsymbol{y}_j$ die Vorhersagen sind und $\hat{\boldsymbol{y}}_j$ die Labels sind) einer gesamten Vorhersage zu berechnen. Siehe Gleichung (1.1).
Der Fehler $\bar{C}(\mathbb{X})$ eines ganzen Datensatzes $\mathbb{X}$, der Groesse p ergibt sich aus dem arithmetischen Mittel der einzelnen Vorhersagen-Fehler. Siehe Gleichung (1.2).

$$C\left(\boldsymbol{y}, \hat{\boldsymbol{y}}\right) = \sum_{i=1}^{m} c(y_i, \hat{y}_i) \tag{1.1}$$

$$\bar{C}(\mathbb{X}) = \frac{1}{p} \sum_{j=1}^{p} C\left(\boldsymbol{y}_j, \hat{\boldsymbol{y}}_j\right) \tag{1.2}$$

Eine Kostenfunktion sollte folgende Eigenschaften aufweisen:

- c ist minimal, wenn $y = \hat{y}$
- c waechst mit $|\boldsymbol{y} - \hat{\boldsymbol{y}}|$
- C ist nach jedem y_n partiell differenzierbar (erklaert in Sektion (1.2.2))

Die wichtigsten Kostenfunktionen, welche in dieser Arbeit verwendet werden, werden nun eingefuehrt.

1.2.1.1 Mittlere quadratische Abweichung

Die bekannteste Kostenfunktion ist die "Mittlere quadratische Abweichung" (engl.: mean squared error). Sie ist definiert folgendermassen definiert:

$$C_{MSE} = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 = \frac{1}{2n}(\hat{\boldsymbol{y}} - \boldsymbol{y})^2 \tag{1.3}$$

Sie erfuellt alle Anforderungen an eine Kostenfunktion:

- Ihr Funktionwert ist 0 und minimal fuer $y = \hat{y}$
- Sie ist proportional zu $(\hat{y} - y)^2$
- Ihre partielle Ableitung nach y lautet: $C' = \frac{1}{n}(y - \hat{y})$

1.2.1.2 Logistische Regression Kostenfunktion

Lossfunction:

$$L_{LG} = -(y \log \hat{y} + (1 - y)\log(1 - \hat{y})) \tag{1.4}$$

Costfunction:

$$J(\boldsymbol{w}, \boldsymbol{b}) = \frac{1}{m}\sum_{i=1}^{m} L(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i) = -\frac{1}{m}\sum_{i=1}^{m}[y_i ...] \tag{1.5}$$

1.2.1.3 Cross-entropy

1.2.2 Gradientenverfahren

Um ein Modell nun zu trainieren, muss man verstehen, dass es sich um ein Optimierungsproblem handelt. Das Modell ist am besten, also macht die besten Vorraussagen, wenn die Funktionswerte der Fehlerfunktion am kleinsten sind. Deshalb muss diese Fehlerfunktion C minimiert werden. Hierbei muss die Funktion C nicht mehr in Abhaengigkeit der Inputs betrachtet werden, sondern in Abhaengigkeit von den Modellparametern $C(\lambda_1, \lambda_2, \ldots, \lambda_n)$, denn diese sollen angepasst werden, um das Modell zu verbessern. Fuer diese Optimiertung wird das sogennante **Gradientenverfahren** (engl.: Gradient descent) verwendet. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \ldots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ (wie z.B. die Fehlerfunktion) minimiert werden. Dies geschieht, indem ein Startpunkt (Ortsvektor) $\boldsymbol{p}_0$, dessen Komponenten den Input von $f(\boldsymbol{p}_0)$ darstellt, gewaehlt wird. Nun werden iterativ neue Punkte $\boldsymbol{p}_t$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte, die den Funktionswert $f(\boldsymbol{p}_t)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genug nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\boldsymbol{b}_t$ bestimmt werden, welcher auf den Punkt $\boldsymbol{p}_t$ draufaddiert einen neuen Punkt $\boldsymbol{p}_{t+1}$ bildet, bei dem der Funktionswert $f(\boldsymbol{p}_{t+1})$ kleiner ist als der von $f(\boldsymbol{p}_t)$. Dies geschieht am effizientesten, wenn $\boldsymbol{b}_t$ in die Richtung der staerksten Funktionswertabnahme zeigt.
Hierzu braucht man den sogennanten **Gradient $\boldsymbol{\nabla}$**, wofür man wiederum partielle Ableitungen braucht.

[1] Im Gymnasium wird beigebracht die lokalen Extrema (inkl. lokalen Minimas) zu bestimmen, indem die erste Ableitung f' gebildet wird und gleich null gesetzt wird. Dies ist hier nicht moeglich, da die Funktion $C'(\lambda_1, \lambda_2, \ldots, \lambda_n)$ deutlich zu komplex ist, um die Nullstellen analytisch zu bestimmen. Deshalb wird das Gradientenverfahren verwendet.

Partielle Ableitungen

Partielle Ableitungen sind eine Erweiterung der "normalen" Ableitungen auf multidimensionale Funktionen $f(x_1, \ldots, x_n)$. Man leitet dabei nur nach einem Argument x_i ab und betrachtet die restlichen Argumente als Konstanten. Es gelten die gleichen Ableitungsregeln wie bei der nicht-partiellen Ableitung. Die partielle Ableitung einer Funktion $f(x_1, \ldots, x_n)$ bezueglich der Variable x_i in einem Punkt $\boldsymbol{a} = (a_1, \ldots, a_n)^\mathsf{T}$ ist analog zur "normalen" Ableitung folgendermassen definiert:

$$\frac{\partial f}{\partial x_i}(\boldsymbol{a}) := \lim_{h \to 0} \frac{f(a_1, \ldots, a_i + h, \ldots, a_n) - f(a_1, \ldots, a_i, \ldots, a_n)}{h}$$

Geometisch ist dies die Steigung der Tangente im Punkt $\boldsymbol{a}$ in Richtung der Achse des Parametern x_i, nach dem man ableitet.

Gradient

Der Gradient $\boldsymbol{\nabla}$ ist ein Differentialoperator, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet.
Dabei fasst man alle partiellen Ableitungen einer Funktion f, nach jedem Argument x_i in einem Vektor zusammen.

$$\boldsymbol{\nabla} f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Geometisch ist der Gradient $\boldsymbol{\nabla} f(\boldsymbol{p})$ einer Funktion f in einem Punkt $\boldsymbol{p}$ dann der Vektor, welcher in die Richtung des steilsten Anstiegs von f zeigt. Sein Betrag gibt die Staerke des Anstiegs an.

Abbildung 1.3: Skalarfeld mit zugehoerigem Gradientenfeld

Da der Gradient in die Richtung des steilsten Anstiegs zeigt, sollte der Vektor $\boldsymbol{b}_t$ in die Richtung des negierten Gradient der Funktion f im Punkt $\boldsymbol{p}_t$ zeigen. Also kann jetzt das iterative Annahern an das lokalen Minimums so beschrieben werden (wobei η eine spaeter erklaerte Schrittgroesse darstellt):

$$\boldsymbol{p}_{t+1} = \boldsymbol{p}_t - \eta \cdot \boldsymbol{\nabla} f(\boldsymbol{p}_t) \tag{1.6}$$

Waehrend des Gradientenverfahren konvergiert der Punkt $\boldsymbol{p}_t$ zu einem belibigen *lokalen* Minimum, abhängig davon wie der Startpunkt $\boldsymbol{p}_0$ gewaehlt wurde. Da dieser meist zufaellig bestimmt wird, ist es eine Glückssache ein sehr niedriges Minimum zu finden.

Die sogennante **Lernrate** η aus Gleichung (1.6) ist ein Hyperparameter. Sie ist ein positiver Proportionalitatsfaktor, welcher die Schrittgroesse des Gradientenabstieg bestimmt. Sie muss je nach zu minimierender Funktion anderst gewaehlt werden. Dabei hilft nur ausporbieren. Falls η nicht gut gewahelt wurde, gibt es Probleme beim Training:

- Falls η zu klein ist, verlaueft das Trainings unnotig langsam und braucht sehr lange. Ausserdem kann es passieren, dass man bei einem hohen lokalen Minimum stecken bleibt.

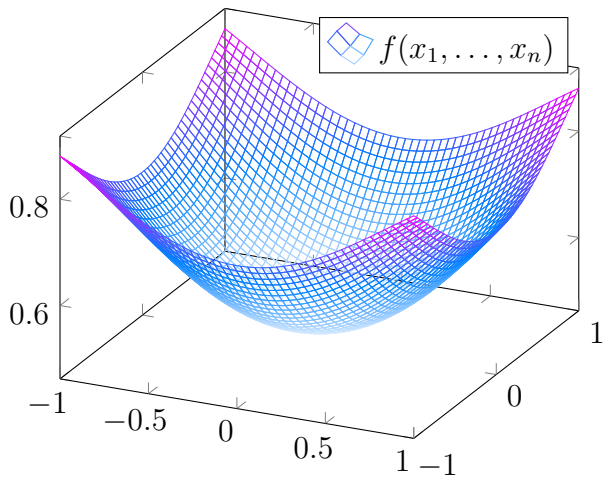


Abbildung 1.4: Visualisierung des Gradientenabstiegs

- Falls η zu gross ist, passiert es, dass man ueber das lokale Minimum hinaus schiesst und nur darum herum springt.

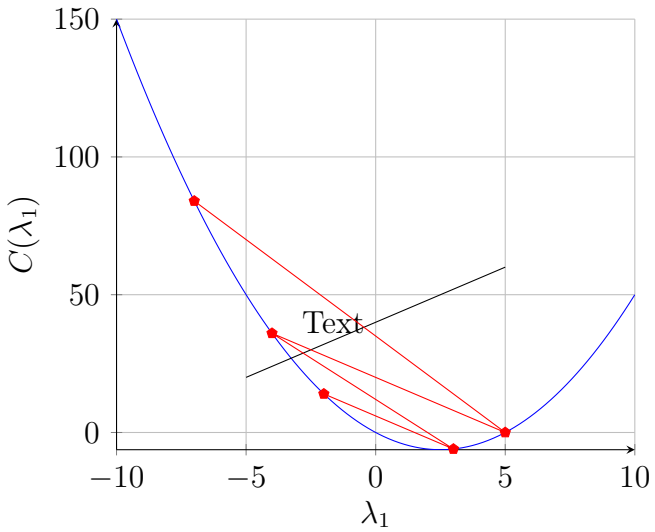


Abbildung 1.5: Visualisierung verschiedener Lernraten

1.2.3 Stochastisches Gradientenverfahren fuer Machine Learning

Wie vorhin erklaert, wird fuer das Trainieren eines Modells das Gradientenverfahren benutzt. Konkret wird die Kostenfunktion $C(\boldsymbol{y}, \hat{\boldsymbol{y}} \mid \lambda_0, \ldots, \lambda_n)$ (λ sind die Modellparameter, $\boldsymbol{y}$ die Vorhersagen und $\hat{\boldsymbol{y}}$ die Labels) minimieren, indem die Parameter angepasst werden. Dadurch macht das Modell immer bessere Vorhersagen. Fuer nur eine Iteration des Gradientenverfahren, muesste man den Gradienten fuer den *gesamten* Trainingsdatensatz berechnen. Dies waere zwar ein exakter Prozess, aber ein extrem langsamer zugleich. Bei grossen Datensaetzen wuerde es eine Ewigkeit dauern, bis das Modell nur annahernd guete Vorhersagen machen wuerde.

Aus diesem Grund verwendet man eine leicht abgeänderte Variante dieses Verfahren, naemlich das **Stochastische Gradientenverfahren** (engl.: Stochastic Gradient Descent) (SGD). Hierfuer wird der "echte" Gradient des gesamten Datensatzes mit dem Gradienten einiger Trainingsbeispiel approximiert. Dazu wird der Trainingsdatensatz in sogennante **Mini-Batches**

eingeteilt und der Gradient jeweils pro Mini-Batch berechnet. Als Konsequenz finden deutlich mehr Iterationen statt in *einer* Durchkaemmung der Trainingsdaten, welche man als **Epoche** bezeichnet. Oft wird mehrere Epochen lang trainiert. Der Gradient eines genug grossen Mini-Batches ist zwar nicht ganz exakt, aber approximiert den Gradieten des gesamten Datensatzen genuegend gut. Sowohl die Mini-Batch Groesse, wie auch die Anzahl Epochen sind weitere Hyperparameter.

Die partiellen Ableitungen der gesamten Trainingsdaten wird mit dem arithmetischen Mittel der partiellen Ableitungen eines Mini-Batches der Groesse q approximiert. Siehe Gleichung (1.7).

$$\frac{\partial \bar{C}}{\partial \lambda_k} \approx \frac{1}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_k} \tag{1.7}$$

Eine Iteration des Stochastischen Graidentenverfahren wird analog zu Gleichung (1.6) folgendermassen durchgefuehrt:

$$\lambda_{k,t+1} = \lambda_{k,t} - \frac{\eta}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_{k,t}} \tag{1.8}$$

1.2.4 Adam Optimizer

ZUSAMMENFASSUNG TRAININGSALGORITHMUS

1.3 Trainingsphaenomene

1.3.1 Overfitting

Kapitel 2

Deep Learning und Künstliche Neuronale Netze

Das wohl beste Modell fuer die meisten Problemstellungen in Bereich Machinelles Lernen (Bilderkennung, Spracherkennung, etc.) ist das **Kuenstliche Neuronale Netz**, kurz KNN (eng.: Neural Network). Die Methoden welche benutzt werden um Neuronale Netze zu traniert fasst man unter dem Begriff **Deep Learning** zusammen.
Kuenstliche Neuronale Netze sind zum Teil biologisch inspiriert von Nervensystemen von Lebewesen. Sie sind aber lediglich eine Abstraktion der Informationverarbeitung und versuchen nicht eine moeglichst genaue biologische Abbildung darzustellen. Es gibt nicht nur eine Art von Neuralem Netz, sondern es exisitieren die verschiedensten Architekturen, welche je nach Problemstellung ausgewaehlt werden muessen. Diese Arbeit wird sich spaeter vorallem mit sogennanten Autoencodern auseinandersetzen.

2.1 Perzeptron

Um den Aufbau und die Funktion eines Kuenstlichen Neuronalen Netz besser zu verstehen, wird im folgenden ein Vorgaenger des KNN erklaert: das **Perzeptron**.

Das einlagige Perzeptron wurde erstmals 1958 von Frank Rosenblatt vorgestellt. Dieses besteht aus einem einzigen Kuenstlichen Neuron. Dieses Kuenstliche Neuron hat mehrere binaere Inputs und einen einzigen binaeren Output. Binaer bedeutet, dass der Wert nur entweder 0 (*aus*) oder 1 (*ein*) sein kann. Des weiteren besitzt es mehrere sogenannte **Gewichte** $w_1, \ldots, w_n \in \mathbb{R}$, fuer jeden Input ein Gewicht. Diese sind reelle Zahlen, welche das Verhalten des Perzeptron bestimmen. Die **gewichtete Summe**, also die Summe aller Produkte der Inputs mit ihrem Gewicht, wird mit $\tilde{z}$ bezeichnet. Sie ist das gleiche wie das Skalarprodukt des Gewichtevektor mit dem Inputvektor:

$$\tilde{z} = \sum_{i=1}^{n} w_i x_i = \boldsymbol{w} \cdot \boldsymbol{x}$$

Zusaetzlich besitzt das Perzeptron einen **Schwellenwert** $\tilde{b}$. Zusammen mit den Gewichten, bildet er die Modellparameter. Das Perzeptron verhaelt sich so, dass falls die gewichtete Summe groesser als der Schwellwert ist, das Neuron feuert, d.h. der Output betraegt 1. Andernfalls ist er 0. Siehe erster Teil der Gleichung (2.1). Es ist gaengig die Ungleichung der Bedingung in die Nullstellenform zu bringen und $\tilde{b}$ durch die **Neigung** (engl.: Bias) $b = -\tilde{b}$ zu ersetzten. Somit lautet die Ungleichung: $\tilde{z} + b > 0$. Der Term $\tilde{z} + b$ wird mit z bezeichnet. Siehe Rest der Gleichung (2.1). Die Neigung gibt an wie stark das Neuron dazu neigt zu feuern. Ein hohes b

laesst ein Neuron auch trotz einigen Nullen in den Inputs feuern, waehrend es fuer ein tiefes b nicht feuern lassen wuerde.

$$p(\boldsymbol{x}) = \begin{cases} 1 & \text{falls } \tilde{z} > \tilde{b} \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \tilde{z} + b > 0 \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \boldsymbol{w} \cdot \boldsymbol{x} + b > 0 \\ 0 & \text{ansonsten} \end{cases} \tag{2.1}$$

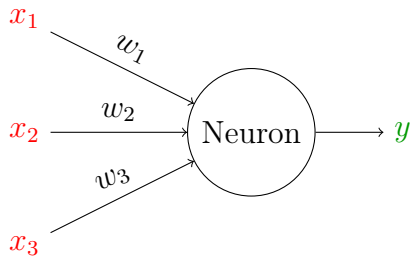


Abbildung 2.1: Perzeptron mit drei Inputs

Fuer das Trainieren des Perzeptron existieren spezielle Verfahren, welche hier aber nicht relevant sind. Das Gradientenverfahren kann naemlich nicht verwendet werden. Der Grund dafuer sollte spaeter in Sektion (2.2) einleuchtend werden.

2.1.1 Was kann ein Perzeptron erlernen?

Nun stellt sich die Frage, was ein Perzeptron erlernen kann und wofuer es genutzt werden kann. Das Perzeptron ist lediglich ein **Linearer Klassifikator** der Form $y = w_1x_1 + \cdots + w_nx_n$. Es kann die Features in zwei Klassen 0 oder 1 einordnen. Ueberschreitet y den Schwellenwert, werden die Features der Klasse 1 zugeordnet, sonst der Klasse 0. Jedoch muessen diese Klassen linear separierbar sein. Das bedeutet, dass die Featurevektoren $\boldsymbol{x}$ durch eine Hyperebene trennbar sein muessen. Bei zwei Features (zwei Inputs) handelt es sich bei der Hyperebene um eine Gerade welche die Vektoren auftrennt. Siehe Abbildung (2.2). Fuer drei Featuers ist es eine Ebene. ZU ABSTRAKT -> BEISPIEL
Eine Funktionen, welche nicht linear separierbar ist (z.B XOR-Verknuepfungen), kann von einem Perzeptron nicht erlernt werden.

Abbildung 2.2: erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts)

(Mehrere, *Perzeptron — Wikipedia, Die freie Enyklopädie*)

2.1.2 Biologische Analogie

SCHREIBEN

Abbildung 2.3: Nervenzelle

2.2 Erweiterung der Kuenstlichen Neuronen

Ein Perzeptron ist, wie vorhin erklaert, nur in der Lage, lineare Klassifikationen durchzufuehren. Um nun auch kompliziertere Probleme zu loesen, muss das Prinzip ausgebaut werden. Ausserdem brauchen wir ein Kuenstliches Neuron, welches sich besonders gut als Baustein fuer KNNs eignet.

2.2.1 Kuenstliche Neuronen im Allgemeinen

Kuenstliche Neuronen sind immer so aufgebaut, dass sie einen oder mehrere Inputs haben und einen einzigen Output. Zu jedem Input x_i ist ein Gewicht w_{ji} assoziert. Zuerst wird die gewichtete Summe der Inputs $\tilde{z}$ gebildet. Die Neigung b wird ebenfalls draufaddiert, um z zu erhalten. Nun muss die sogennante Aktivierung a gebildet werden. Sie entspricht dem Output des Neurons. Die Aktivierung a ist das Resultat der Aktivierungsfunktion φ angewendet auf z. $a = \varphi(z)$. Die verschiedenen Kuenstlichen Neuronen unterscheiden sich fast nur in ihrer Aktivierungsfunktion.

Abbildung 2.4: ein kuenstliches Neuron und seine Bestandteile

2.2.2 Perzeptronen als Kuenstliche Neuronen

Zuerst noch einmal ein Blick auf das Perzeptron im Angesicht der Aktivierungsfunktion. Ein wesentlicher Unterschied des Perzeptron gegenueber sonstigen Kuenstlichen Neuronen, besteht darin, dass seine Inputs und Outputs nur binaere Werte annehmen koennen. Um dieses Verhalten des Perzeptron zu erhalten, muss eine Stufenfunktion als Aktivierungfunktion verwendet werden: die Heaviside-Funktion Θ. Sie hat einen einzigen Stufensprung bei $x = 0$ vom Wert 0 auf 1. Siehe Abb. (2.5).

$$\varphi^{\text{hlim}}(z) = \Theta(z) = \begin{cases} 1 & \text{falls } z \geq 0 \\ 0 & \text{falls } z < 0 \end{cases}$$

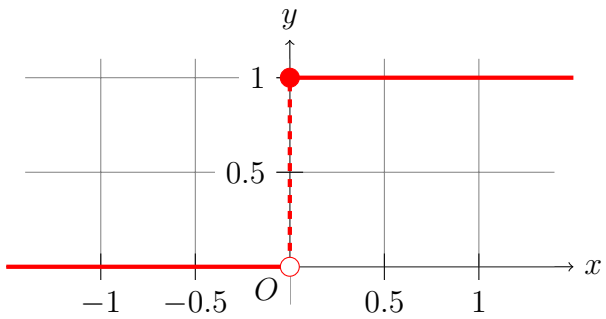


Abbildung 2.5: Definition und Graph der Heaviside-Funktion Θ

2.2.3 Stueckweise lineare Neuronen

Der naechste Schritt nach dem binaeren Perzeptron sind sogenannte stueckweise lineare Neuronen. Sie verwenden eine stueckweise lineare Aktivierungsfunktion. Diese bildet ein beschraenktes Intervall linear ab. Die Werte ausserhalb werden auf die konstanten Werte 0 oder 1 abbgebildet. Siehe Abb. (2.6).

Die Inputs koennen jetzt beliebige reelle Zahlen sein (vorzugsweise in der Naehe von 0 und 1). Ein KNN aus ausschliesslich linearen Neuronen ist ziemlich nutzlos, da jedliche Verkettung von linearen Funktion auch durch eine einzige lineare Funktion dargestellt werden koennte. Somit

hat ein KNN gegenueber einem einzelnen Neuron keinen Mehrwert. Ausserdem koennen mit diesen Neuron auch nur lineare Probleme geloest werden.

$$\varphi^{\text{pwl}}(z) = \begin{cases} 1 & \text{falls } z > \frac{1}{2} \\ z + \frac{1}{2} & \text{falls } -\frac{1}{2} < z < \frac{1}{2} \\ 0 & \text{falls } z < \frac{1}{2} \end{cases}$$

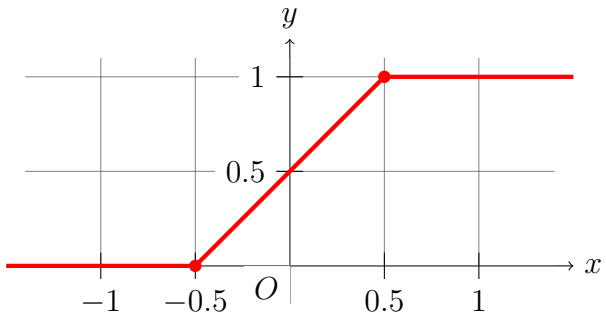


Abbildung 2.6: Formel und Graph einer stueckweisen linearen Funktion

2.2.4 Sigmoide Neuronen

Die logische naechste Erweiterung sind nicht-lineare Neuronen. Ein sehr beliebter Kandidat dafuer sind Sigmoide Neuronen. Den Namen haben sie von ihrer Aktivierungsfunktion: der Sigmoidfunktion σ. Eine sehr wichtige Eigenschaft von ihr ist, dass sie - im Gegensatz zu den vorhin genannten Aktivierungsfunktion - ueberall differenzierbar und strikt monoton steigend ist. Erst fuer diese Aktivierungsfunktion, kann das Gradientenverfahren angewendet werden und somit das KNN trainiert werden. Die anderen Funktionen haben Stellen bei welchen die Ableitungsfunktion null betraegt, somit kann das lokale Minimum nicht gefunden werden. Nicht nur die Sigmoidfunktion erfuellt diese Bedingungen, sondern auch andere. Die Sigmoidfunktion wird bevorzugt, da ihre Ableitung sehr simpel ist, siehe Abb. (2.7). Die Nicht-linearitaet dieser Neuronen ermoeglicht ein Erlernen von deutlich komplexeren Sachverhalten. Und wie spaeter in Sektion (2.6) erlautert, ist es moeglich mit der Kompostion von nicht-linearen Funktionen jede belibige Funktion zu approximieren!

Die Sigmoid-Funktion besitzt eine einzige Wendestelle $\sigma''(x = 0) = 0$ und hat zwei Asymptoten, eine $\lim\limits_{x \to -\infty} \sigma(x) = 0$ und eine zweite $\lim\limits_{x \to \infty} \sigma(x) = 1$. Siehe Abb. (2.7).

$$\varphi^{\text{sig}}(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

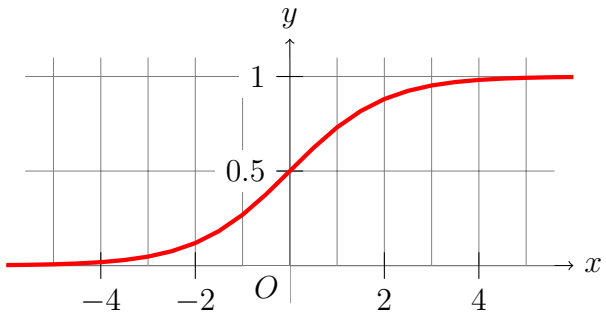


Abbildung 2.7: Definition, Ableitung und Graph der Sigmoid-Funktion σ

2.3 Topologie der Kuenstlichen Neuronalen Netzen

Nun sollten diese Sigmoiden-Neuronen als Bausteine verwendet werden, um ein Kuenstliches Neuronales Netz zu bilden. Dazu werden sie miteinander verbundet und bilden so ein Netz, aehnlich wie ein Nervensystem. Diese Neuronen sind in verschieden Schichten (engl.: Layers)

arangiert. Die erste ist die **Inputschicht**. Sie behinhalten die Inputneuronen. Dies sind eigentlich keine richtigen Neuronen sondern eher Platzhalter fuer ihren jeweiliges Feature x_i. Als letztes kommt die **Outputschicht** mit den Outputneuronen, welche jeweils einen Outputwert y_i besitzen. Dazwischen liegen die **Zwischenschichten** (engl.: Hiddenlayer). Von ihnen kann es beliebig viele geben und in ihnen beliebig viele Neuronen. Falls viele Zwischenschichten verwendet werden, bezeichnet man das Netzwerk als "deep". Daher ruert auch der Name des Deep Learnings. Der Aufbau eines KNN bezeichnet man als **Topologie** des Netzes. Die Topologie umfasst viele Hyperparameter. Darunter sind zum Beispiel die Anzahl Zwischenschichten, wie auch die Anzahl Neuronen pro Schicht.

Jedes Neuron aus einer Schicht ist mit jedem Neuron aus der naechsten Schicht ueber Verbindungen gekoppelt. Alle Verbindungen besitzten ein Gewicht analog zu den Inputs des Perzeptron. Die Aktivierung, also der Output, eines Neurons wandert entlang den jeweiligen Verbindung zu allen Neuronen der naechsten Schicht und dient als deren Input. Die soeben beschriebe Art von KNN nennt man **Feedforward-Netz**, da alle Werte ausschliesslich nach vorne propagiert werden.

In Abbildung (2.8) ist ein Beispiel eines Neuronalen Netzes abgebildet. In diesem Fall besitz es sowohl 4 Inputs, wie auch 4 Outputs. Es hat ausserdem 3 Zwischenschichten. Die zwei aueserren Zwischenschichten haben 3 Neuronen und die dazwischen hat sogar 4 Neuronen.

Die Hoffnung beim Trainieren von KNNs besteht darin, dass das Modell fuer jede weiter Schicht eine hoeheres Abstraktionsniveau erreicht. Wuerde man zum Beispiel ein Netzwerk zur Gesichtserkennung trainieren, koennte man sich den Erkennungsprozess folgendermassen vorstellen. Die erste Zwischenschicht erkennt Kanten und Konturen. Die zweite vereint diese Merkmale zu Ecken und primitive geometrische Formen. Die dritte Schicht sollte dann schon komplexere geometrische Formen erkennen, welche gewissen Gesichtmerkmalen, wie der Nase, aehneln. Die letzten Schichten sollte dann all diese Merkmale zusammensetzen und so ein Gesicht erkennen. SCHICHTENBESCHRIFTUNG HINZUFUEGEN; BIAS ALS EIGENES NEURON?

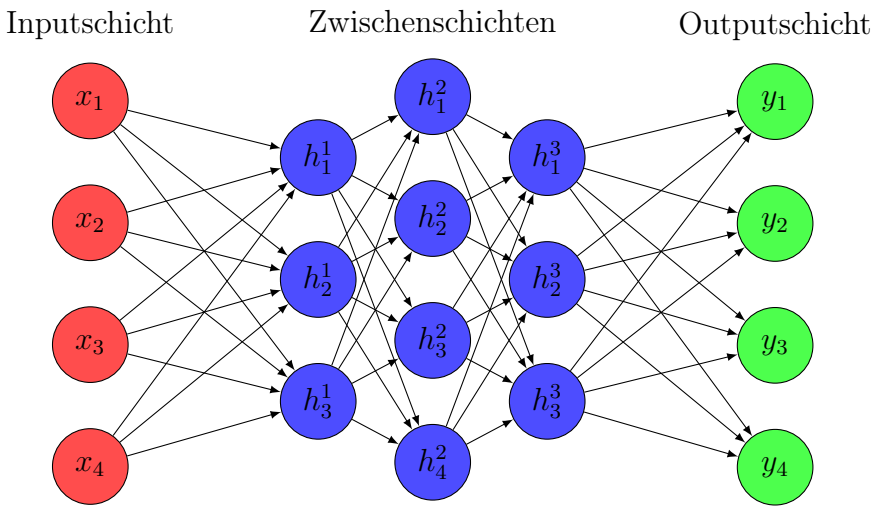


Abbildung 2.8: Schichten eines KNNs

2.4 Vorwaertspropagierung

Jetzt, da der Aufbau eines KNNs erklaert wurde, sollte nun die mathematische Funktionsweise des Modells erklaert werden. Hierfuer muessen einige Konventionen zur Bezeichnung der Teile eines KNNs getroffen werden:

- n_j^l ist das j-te Neuron in der l-ten Schicht.
- z_j^l ist die gewichtete Summe der Inputs des j-ten Neuron in der l-ten Schicht.
- a_j^l ist die Aktivierung des j-ten Neurons in der l-ten Schicht.
- b_j^l ist die Neigung fuer das j-te Neuron in der $(l+1)$-ten Schicht ausgeht. [1]
- $w_{j,k}^l$ ist das Gewicht der Verbindung vom k-ten Neuron in der l-ten Schicht zum j-ten Neuron in der $(l+1)$-ten Schicht. [2]
- L soll die gesamte Anzahl der Schichten sein.
- $|l|$ ist die Anzahl Neuronen in der l-ten Schicht. [3]
- φ ist die gewaehlte Aktivierungsfunktion (grundsaetzlich ist diese immer die Sigmoidfunktion σ).

ABBILDUNG MIT NEURONBESTANTEILEN (GEWICHTEBESCHRIFTUNGEN UND NEIGUNGEN)
DIE GEWICHTBESCHRIFTUNGEN IN ABBILDUNG 11 sind falsch! Die Gewichtsindizes beginnen mit 0 und ended mit L-1

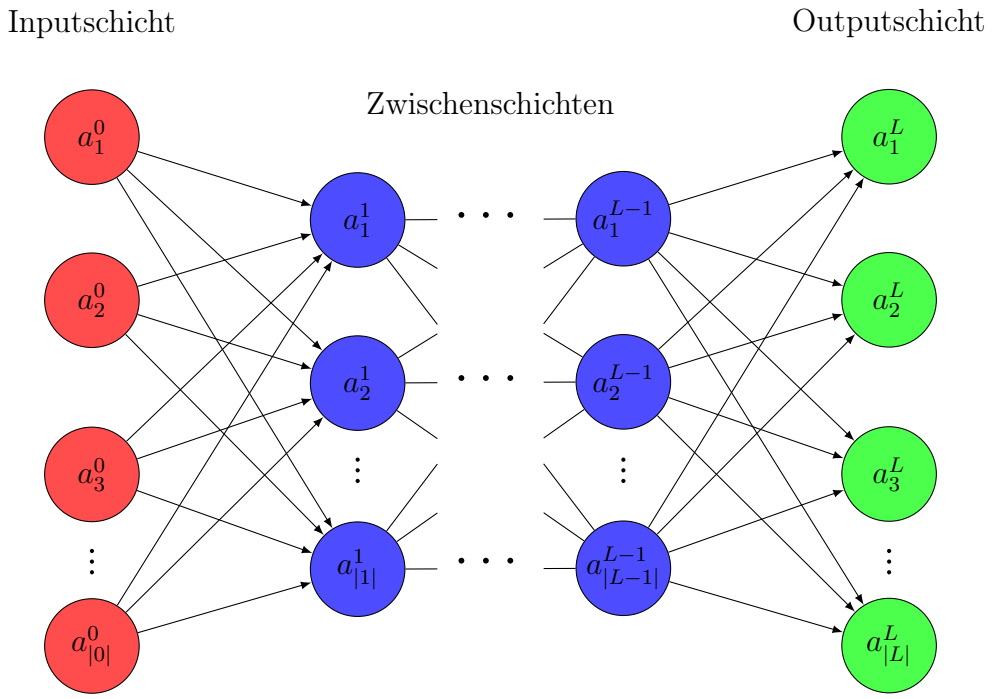


Abbildung 2.9: zum Verstaentniss der Nomenklatur

Abbildung 2.10: Neuron mit Gewichtbeschriftungen und Neigung

[1] Diese Konvention wurde gewaehlt, damit die folgenden Gleichungen simpler sind.

[2] Man beachte die Reihenfolge!
Diese Konvention scheint zwar auf den ersten Blick unintuitiv, macht jedoch Sinn fuer die spaeteren mathematischen Operationen mit Matrizen in Sektion (2.5).

[3] Diese Schreibweise hat nichts mit dem Betrag zu tun, sondern wird einfach gewaehlt, da sie sehr platzsparend ist.

Die Vorwaertspropagierung beginnt bei den Inputneuronen, welche jeweils einen Inputwert in sich tragen. Diese werden, um fuer eine kohaerente Nomenklatur zu sorgen, analog zu den Aktivierungen der anderen Neuronen mit a_j^0 bezeichnet, wobei j der Index des Neurons ist.
Nun muessen die restlichen Aktivierungen der Neuronen berechnet werden. Dies geschieht rekursiv, anhand der Aktivierungen der voherigen Schicht. Und zwar folgendermassen (ersichtlich in Gleichung (2.2)):
Zuerst laueft eine Summe ueber alle Neuronen n_k^l der jetztigen Schicht l. Dabei wird die gewichtete Summe der Aktivierungen a_k^l mit den assozierten Gewichten $w_{j,k}^l$ gebildet. Hierbei ist das Gewicht jenes, welches das k-te Neuron der l-ten Schicht mit dem j-ten Neuron der $(l+1)$-ten Schicht verbindet. Zusaetzlich gehoert zu der gewichteten Summe auch die jeweilige Neigung b_j^l, welche dazuaddiert wird. Diese gewichtete Summe wird mit z_j^{l+1} bezeichnet.

$$z_j^{l+1} = \sum_{k=1}^{N_l} w_{j,k}^l a_k^l + b_j^l \tag{2.2}$$

Auf diese Summe wird dann die Aktivierungsfunktion φ angewandt. Das ist dann die Aktivierung a_j^{l+1} des j-ten Neurons in der $(l+1)$-ten Schicht.

$$a_j^{l+1} = \varphi\left(\sum_{k=1}^{N_l} w_{j,k}^l a_k^l + b_j^l\right) = \varphi\left(z_j^{l+1}\right) \tag{2.3}$$

Da KNNs weit verbreitet sind in der Bilderkennung und Spracherkennung, ist es nicht unueblich, dass diese sehr viele Neuronen und Verbindungen (ueber 100'000) besitzen. Um hierbei nicht den Ueberblick zu verlieren und um nicht in den Indizes zu ertrinken, greift man auf **Lineare Algebra** zurueck. Man verwendet Matrizen und Vektoren um die vielen Variabeln zusammenzufassen. Ausserdem besteht ein weiterer Vorteil darin, dass Computer mithilfe von Vektor- und Matrixoperationen die Berechnungen parallisieren koennen und in kuerzerer Zeit und mit weniger Ressourcen viele Brechnungen gleichzeitig ausfuehren koennen. Dies beschleunigt das Training der Modelle um ein Vielfaches. Dies wird spaeter in Sektion (**??**) noch weiter thematisiert.

Die Inputs $\boldsymbol{x}$, Outputs $\boldsymbol{y}$ und Labels $\hat{\boldsymbol{y}}$ haben wir schon von Anfang an als Vektoren geschrieben. Nun sollen noch die Parameter und die restlichen Komponenten eines KNNs als Vektoren und Matrizen geschrieben werden. Sowohl alle gewichteten Summen z_j^l, wie auch alle Aktivierungen a_j^l einer Schicht l, werden in Vektoren $\boldsymbol{z}^l$ und $\boldsymbol{a}^l$ zusammengefasst. Alle Neigung b_j^l fuer eine Schicht $(l+1)$ bilden ebenfalls eines Vektor $\boldsymbol{b}^l$. Zu guter letzt, wird noch eine **Gewichtsmatrix** $\boldsymbol{W}^l \in \mathbb{R}^{|l+1|\times|l|}$ definiert. Sie enthaltet alle Gewichte welche die l-te Schicht *zu* der $(l+1)$-ten Schicht verbindet. Das heisst der Eintrag in der j-ten Zeile und in der k-ten Spalte ist $w_{j,k}^l$ und verbindet so das Neuron n_k^l zu dem Neuron n_j^{l+1}.

$$\begin{aligned}
\boldsymbol{z}^l &= \begin{pmatrix} z_1^l & z_2^l & \cdots & z_{|l|}^l \end{pmatrix}^\mathsf{T} \\
\boldsymbol{a}^l &= \begin{pmatrix} a_1^l & a_2^l & \cdots & a_{|l|}^l \end{pmatrix}^\mathsf{T} \\
\boldsymbol{b}^l &= \begin{pmatrix} b_1^l & b_2^l & \cdots & b_{|l+1|}^l \end{pmatrix}^\mathsf{T}
\end{aligned}$$

$$\boldsymbol{W}^l = \begin{pmatrix} w_{1,1}^l & w_{1,2}^l & \cdots & w_{1,|l|}^l \\ w_{2,1}^l & w_{2,2}^l & \cdots & w_{2,|l|}^l \\ \vdots & \vdots & \ddots & \vdots \\ w_{|l+1|,1}^l & w_{|l+1|,2}^l & \cdots & w_{|l+1|,|l|}^l \end{pmatrix}$$

Mit diesen Definitionen kann nun Gleichung (2.2) in Matrixform geschrieben werden, denn die Matrixmultiplikation von $\boldsymbol{W}^l$ mit $\boldsymbol{a}^l$ ergibt einen Vektor $\tilde{\boldsymbol{z}}^{l+1}$, welcher alle gewichteten Summen $\tilde{z}_j^{l+1}$ ohne die jeweilige Neigung enthaelt.
STIMMT DIESE GLEICHUNG? (GLAUBE SCHON)

$$\boldsymbol{W}^l\boldsymbol{a}^l = \begin{pmatrix} \sum_{i=1}^{|l|} w_{i,1}^l a_i^l & \sum_{i=1}^{|l|} w_{i,2}^l a_i^l & \cdots & \sum_{i=1}^{|l|} w_{i,|l+1|}^l a_i^l \end{pmatrix}^\mathsf{T} = \tilde{\boldsymbol{z}}^{l+1}$$

Nun muessen wir noch den Neigungsvektor $\boldsymbol{b}^l$ draufaddieren und wir erhalten Gleichung (2.4), mithilfe dessen wir den Vektor der gewichteten Summen $\boldsymbol{z}^{l+1}$ bilden koennen.

$$\boldsymbol{z}^{l+1} = \mathbf{W}^l\boldsymbol{a}^l + \boldsymbol{b}^l \tag{2.4}$$

Der letzte Schritt besteht noch darin die Aktivierungsfunktion auf $\boldsymbol{z}^{l+1}$ anzuwenden um den Aktivierungsvektor $\boldsymbol{a}^{l+1}$ zu bilden. Hierfuer muss aber noch ein neues mathematisches Konzept eingefuehrt werden: die Vektorisierung einer Funktion.

Vektorisierung einer Funktion

Die Vektorisierung einer Funktion f, geschrieben als $\boldsymbol{f}[\boldsymbol{v}]$ ist eine neue Funktion, welche als Rueckgabewert einen Vektor hat, auf dessen Komponenten jeweils immer die Funktion f angewendet wird.

$$\boldsymbol{f}[\boldsymbol{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

Nun kann ganz einfach die vektorisierte Aktivierungsfunktion $\boldsymbol{\varphi}$ auf $\boldsymbol{z}^{l+1}$ angewandt werden.

$$\boldsymbol{a}^{l+1} = \boldsymbol{\varphi}\left[\boldsymbol{W}^l\boldsymbol{a}^l + \boldsymbol{b}^l\right] = \boldsymbol{\varphi}\left[\boldsymbol{z}^{l+1}\right] \tag{2.5}$$

(Nielsen, *Neural Networks and Deep Learning*)

2.4.1 Modellparameter initalisieren

Ein Schritt der getan werden muss, bevor das Training beginnt, ist das Initalisieren aller Modellparameter, in diesem Fall die Gewichte und Neigungen. Dies ist ein sehr essentieller Schritt, denn diese Startwerte entscheiden erheblich ueber die Leistungsfaehigkeit des Modells.

Wie in Sektion (1.2.2) gezeigt, muss am Anfang des Gradientenverfahrens ein Startpunkt $\boldsymbol{p}_0$ innerhalb des Gradientenfeldes $\boldsymbol{\nabla}$ gewaehlt werden, von welchem aus der Gradientenabstieg beginnt. Da wir das Gradientenverfahren zur Optimierung der Modellparameter verwenden, ist unser Startpunkt, der Vektor aller Hyperparameter $\boldsymbol{\lambda} = \begin{pmatrix}\lambda_1 & \cdots & \lambda_n\end{pmatrix}^\mathsf{T}$. Dieser Startpunkt entscheidet darueber, in welches lokale Minimum konvergiert wird und bestimmt somit auch die bestmoegliche Exaktheit der Vorhersagen. Falls schlecht Initalwerte gewaehlt werden, konvergiert der Punkt in ein hohes lokales Minimum, was grosse Kostenfunktionswerte und schlecht Vorhersagen verursacht.

Es ist nicht moeglich im Vorhinein zu wissen, welche Initalwerte gute Resultate liefern. Man muss ausprobieren, deshalb initalisert man gaengierweise die Hyperparameter mit Zufallswerten. Dafuer nimmt man aber nicht irgendwelche Zufallsvariablen sondern man benutzt die Gauss'sche Normalverteilung $\mathcal{N}(\mu, \sigma^2)$ bzw. ihre Dichtefunktion

$$\phi(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}$$

Abbildung 2.11: Graph der Dichtefunktion $\phi(x \mid \mu = 0, \sigma^2 = 1)$ mit $X \sim \mathcal{N}(0, 1)$

Um die Neigungen $b_{t=0}$ zu initalisieren benutzt man eine Normalverteilung $\mathcal{N}(0, 1)$ mit Erwartungswert $\mu = 0$ und Varianz $\sigma^2 = 1$. Um die Gewichte $w_{t=0}$ zu initalisieren benutzt man ebenfalls eine Normalverteilung mit Erwartungswert $\mu = 0$, jedoch wird die Varianz σ^2 so skaliert, dass die Summe aller Zufallesvariabeln der Gewichte einer Schicht eine Varianz von $\sigma^2_{tot} = 1$ hat.

$$w^l_{t=0} \sim \mathcal{N}(\mu = 0, \sigma^2 = \frac{1}{|l|}) \tag{2.6}$$

$$b^l_{t=0} \sim \mathcal{N}(\mu = 0, \sigma^2 = 1) \tag{2.7}$$

WEITERE ERLAUETERUNGEN? SATURATION DER NEURONEN VERHINDERN DESHALB TOTALE VARIANZ NUR 1

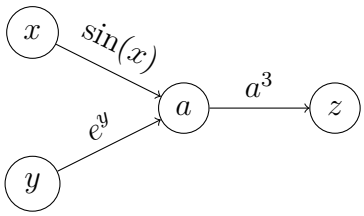
(Mehrere, *Normal distribution — Wikipedia, The Free Encyclopedia*)
(Nielsen, *Neural Networks and Deep Learning*)

2.5 Rueckwaertspropagierung

Die wahre Herausforderung besteht beim Gradientenverfahren darin, die partiellen Ableitungen der Modellparameter, also die Komponenten des Gradienten, zu bestimmen. Anderst gesamt muessen alle Terme $\partial C / \partial w^l_{j,k}$, wie auch alle Terme $\partial C / \partial b^l_k$ berechnet werden. Das Verfahren zum bestimmen dieser Ausdruecke ist so spezfisch und aufwendig, dass das Gradientenverfahren fuer KNNs einen eigenen Namen hat: die **Rueckwaertspropagierung** (engl.: Backpropagation) (auch Fehlerrueckfuehrung).

Da ein KNN, wie der Name es schon sagt, vernetzt ist, koennen die partiellen Ableitungen einer Schicht bezueglich seiner Nachbarsschichten berechnet werden. Dies ist auch der namensgebende Grundgedanke der Rueckwaertsprogagierung: Man beginnt in der letzten Schicht die partiellen Ableitungen zu bestimmen und berechnet dann im Rueckwaertsgang Schicht fuer Schicht die vorherigen partiellen Ableitungen, bis nach vorne.
Dies macht man mithilfe der Kettenregel der Ableitungen. Es ist sinnvoll fuer das Aufstellen dieser Gleichungen das Netzwerk als **Computation Graph** zu betrachten.

Ein Computation Graph ist eine Darstellung einer Verkettung von Funktionen als Netzwerk von Operationen. Die Knoten im Graph stellen Variabeln dar und die Pfade, welche die Knoten verbinden, sind die Funktionen, welche die Variabeln aufeinander abbilden. Die Funktion wird auf die Variable angewandet, von der der Pfad ausgeht. Der Knoten in welchem der Pfad endet, nimmt dann den Funktionswert an. Falls mehrere Pfade in einem Knoten enden, werden die einzelnen Werte der Pfade zusammenaddiert, um die Variable zu bilden.



$$a(x,y) = \sin(x) + e^y$$
$$z(a(x,y)) = a^3(x,y) = (\sin(x) + e^y)^3$$

$$\frac{\partial z}{\partial x} = \frac{\partial a}{\partial x} \cdot \frac{\partial z}{\partial a} = \cos(x) \cdot 3a^2$$
$$\frac{\partial z}{\partial y} = \frac{\partial a}{\partial y} \cdot \frac{\partial z}{\partial a} = e^y \cdot 3a^2$$

Abbildung 2.12: Computation Graph einer exemplarischen Verkettung von Funktionen

In Abbildung (2.12) ist ein Beispiel eines Computation Graphs zusammen mit der Herleitung der partiellen Ableitungen dargestellt.
Der erste Schritt der Rueckwaertsprogagierung besteht darin, dass man die partiellen Ableitungen $^{\partial C}/_{\partial z_j^l}$ der Kostenfunktion C bezueglich den gewichteten Summen z_j^l aller Schichten berechnet. Daraus laesst sich dann spaeter sehr einfach die partiellen Ableitungen bezueglich den Gewichten $^{\partial C}/_{\partial w_{j,k}^l}$ und bezueglich den Neigung $^{\partial C}/_{\partial b_j^l}$ berechnen.

Zur Uebersichtlichkeit definiert man einen **Fehler** δ_j^L fuer jeder j-te Neuron in jeder l-ten Schicht, welcher die partielle Ableitung bezueglich der gewichteten Summe ist. Ebenfalls definiert man analog einen Fehlervektor $\boldsymbol{\delta}^l$, welcher alle Fehler δ_j^l einer Schicht l zusammenfasst. Nun heisst es diesen fuer jedes Neuron jeder Schicht zu berechnen.

$$\delta_j^l := \frac{\partial C}{\partial z_j^l} \tag{BP0}$$

$$\boldsymbol{\delta}^l := \begin{pmatrix} \frac{\partial C}{\partial z_1^l} & \frac{\partial C}{\partial z_2^l} & \cdots & \frac{\partial C}{\partial z_{|l|}^l} \end{pmatrix}^\mathsf{T} \tag{BP0a}$$

Da die Kostenfunktion unmittelbar auf die letzte Schicht angewandt wird, beginnt man auch mit der Berechnung des Fehlers in der letzten Schicht L. Wir stellen nun einen Computation Graph auf, um die partiellen Ableitungen zu bilden.

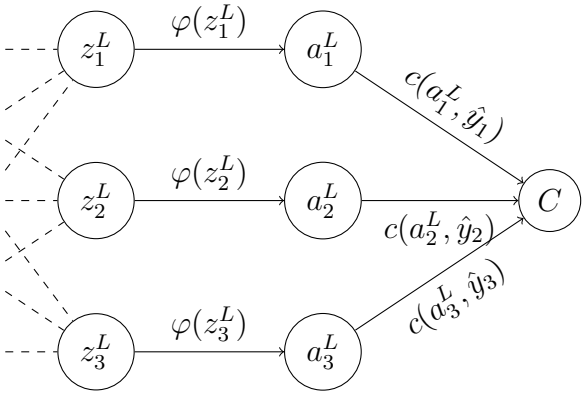


Abbildung 2.13: Computation Graph zur Berechnung von δ_j^L

Wir koennen dem Computation Graph aus Abbildung (2.14) entnehmen, dass die Kosten C eine Funktion in Abhaengigkeit von den letzten Aktivierungen a_j^L ist, welche wiederum in Abhaengigkeit von den gewichteten Summen z_j^L berechnet werden. Somit koennen wir mithilfe

der Kettenregel folgende Beziehung aufstellen:

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial a_j^L}{\partial z_j^L} \cdot \frac{\partial C}{\partial a_j^L}$$

Da a_j^L durch die Anwendung der Aktivierungsfunktion φ auf z_j^L gebildet wrid, ist ${}^{\partial a_j^L}/_{\partial z_j^L}$ einfach die Ableitung der Aktivierungsfunktion $\varphi'(z_j^L)$. Somit koennen wir schreiben:

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \cdot \varphi'(z_j^L) \tag{BP1}$$

Nun moechten wir diese Ausdruecke wieder in Matrixschreibweise realisieren, welche die ganze Schicht L zusammenfasst. Dazu muss eine neue Operation eingefuehrt werden: das Hadamard-Produkt.

Hadamard-Produkt

Das Hadamard-Produkt ist ein spezielles Produkt zweier gleichgrossen Matrizen $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ und $\boldsymbol{B} \in \mathbb{R}^{m \times n}$. Die resultierende Matrix ergibt sich aus der elementweisen Multiplikation der Ausgangsmatrizen.

$$\boldsymbol{A} \odot \boldsymbol{B} = \begin{pmatrix} A_{1,1}B_{1,1} & \cdots & A_{1,n}B_{1,n} \\ \vdots & \ddots & \vdots \\ A_{m,1}B_{m,1} & \cdots & A_{m,n}B_{m,n} \end{pmatrix} \in \mathbb{R}^{m \times n} \qquad \boldsymbol{v} \odot \boldsymbol{w} = \begin{pmatrix} v_1 w_1 \\ \vdots \\ v_n w_n \end{pmatrix}$$

Mit $\boldsymbol{\varphi}'$ als die vektorisierte Ableitung der Aktivierungsfunktion, koenne wir den Fehlervektor der letzten Schicht so berechnen:

$$\boldsymbol{\delta}^L = \begin{pmatrix} \frac{\partial C}{\partial a_1^L} & \frac{\partial C}{\partial a_2^L} & \cdots & \frac{\partial C}{\partial a_{|L|}^L} \end{pmatrix}^{\mathsf{T}} \odot \boldsymbol{\varphi}'(\boldsymbol{z}^L)$$

Dabei ist der erste Operand des Hadamard-Produkt, nichts anderes als der Gradient $\boldsymbol{\nabla}_{\boldsymbol{a}^L} C$ der Kostenfunktion C bezueglich dem Aktivierungsvektor $\boldsymbol{a}^L$ der letzten Schicht. Dieser Gradient kann einfach berechnet werden, indem man die Ableitungsfunktion fuer die gewaehlte Kostenfunktion bildet. Wuerde man die Mittlere quadratischen Abweichung $C = \frac{1}{2}(\hat{\boldsymbol{y}} - \boldsymbol{a}^L)^2$ als Kostenfunktion waehlen, haetten wir $\boldsymbol{\nabla}_{\boldsymbol{a}^L} C = (\boldsymbol{a}^L - \hat{\boldsymbol{y}})$
Daraus folgt folgende kompakte Gleichung fuer den Fehler der ganzen letzten Schicht:

$$\boldsymbol{\delta}^L = \boldsymbol{\nabla}_{\boldsymbol{a}^L} C \odot \boldsymbol{\varphi}'(\boldsymbol{z}^L) \tag{BP1a}$$

Nun muessen wir eine rekursive Berechnungsmethode des Fehlers δ_j^{l-1} der vorherigen Schicht anhand des Fehlers δ_j^l der jetztigen Schicht erarbeiten. Dafuer stellen wir zuallererst wieder einen Computation Graph auf.
Es gilt erneut folgende Gleichung fuer die Berechnung des Fehlers δ_j^{l-1}:

$$\delta_j^{l-1} = \frac{\partial C}{\partial z_j^{l-1}} = \frac{\partial a_j^{l-1}}{\partial z_j^{l-1}} \cdot \frac{\partial C}{\partial a_j^{l-1}}$$

Beim Uebergang der Schicht $(l-1)$ zur Schicht l beinflusst jene Aktivierungen a_j^{l-1} alle gewichtete Summen z_k^l, daher folgt mit der Kettenregel, dass die partielle Ableitung ${}^{\partial C}/_{\partial a_j^{l-1}}$ die Summe aller ${}^{\partial z_k^L}/_{\partial a_j^{l-1}} \cdot {}^{\partial C}/_{\partial z_k^l}$ sein muss.

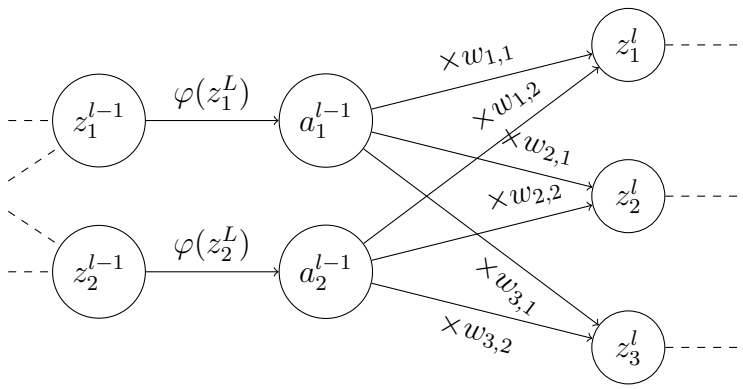


Abbildung 2.14: Computation Graph zur Berechnung von δ_j^{l-1}

Somit erhalten wir:

$$\delta_j^{l-1} = \varphi'(z_j^{l-1}) \cdot \sum_{n=1}^{|l|} \left(\frac{\partial C}{\partial z_n^l} \cdot \frac{\partial z_n^l}{\partial a_j^{l-1}} \right)$$

Da bei der Bildung der gewichteten Summe z_j^l wird die Aktivierung a_k^{l-1} einfach mit der Gewicht $w_{n,j}^{l-1}$ multipliziert. Somit ist das Gewicht gerade die partielle Ableitung. Ausserdem ist der Term $\partial C / \partial z_j^l$ per Definition der Fehler δ_j^l.

$$\delta_j^{l-1} = \varphi'(z_j^{l-1}) \cdot \sum_{n=1}^{|l|} \left(w_{n,j}^{l-1} \cdot \delta_m^l \right) \tag{BP2}$$

Auch diese Gleichung haetten wir gerne in der Matrixschreibweise. Wir beginnen mit der Erweiterung auf alle gewichteten Summen.

$$\boldsymbol{\delta}^{l-1} = \boldsymbol{\varphi}'(\boldsymbol{z}^{l-1}) \odot \begin{pmatrix} \sum_{n=1}^{|l|} w_{1,n}^{l-1} \cdot \delta_n^l & \cdots & \sum_{n=1}^{|l|} w_{|l-1|,n} \cdot \delta_n^l \end{pmatrix}^\mathsf{T}$$

Der zweite Operant des Hadamard-Produkts ist hierbei gerade das Produkt der Matrixmultiplikation zwischen der transponierten Gewichtsmatrix $(\boldsymbol{W}^{l-1})^\mathsf{T}$ der Schicht $(l-1)$ und dem Fehlers $\boldsymbol{\delta}^l$ der Schicht l.
GEWICHTSMATRIX INDIZES STIMMEN WAHRSCHEINLICH NICHT

$$\begin{aligned} \begin{pmatrix} \sum_{n=1}^{|l|} w_{1,n}^{l-1} \cdot \delta_n^l & \cdots & \sum_{n=1}^{|l|} w_{|l-1|,n} \cdot \delta_n^l \end{pmatrix}^\mathsf{T} &= \begin{pmatrix} w_{1,1}^{l-1} & w_{2,1}^{l-1} & \cdots & w_{|l-1|,1}^{l-1} \\ w_{1,2}^{l-1} & w_{2,2}^{l-1} & \cdots & w_{|l-1|,2}^{l-1} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1,|l|}^{l-1} & w_{2,|l|}^{l-1} & \cdots & w_{|l-1|,|l|}^{l-1} \end{pmatrix} \begin{pmatrix} \delta_1^l & \cdots & \delta_{|l|}^l \end{pmatrix}^\mathsf{T} \\ &= \begin{pmatrix} w_{1,1}^{l-1} & w_{1,2}^{l-1} & \cdots & w_{1,|l|}^{l-1} \\ w_{2,1}^{l-1} & w_{2,2}^{l-1} & \cdots & w_{2,|l|}^{l-1} \\ \vdots & \vdots & \ddots & \vdots \\ w_{|l-1|,1}^{l-1} & w_{|l-1|,2}^{l-1} & \cdots & w_{|l-1|,|l|}^{l-1} \end{pmatrix} \boldsymbol{\delta}^l = (\boldsymbol{W}^{l-1})^\mathsf{T} \boldsymbol{\delta}^l \end{aligned}$$

Alles in Allem erhalten wir:

$$\boldsymbol{\delta}^{l-1} = ((\boldsymbol{W}^{l-1})^\mathsf{T} \boldsymbol{\delta}^l) \odot \boldsymbol{\varphi}'(\boldsymbol{z}^{l-1}) \tag{BP2a}$$

Nun gilt es noch die partiellen Ableitungen fuer die Gewichte und die Neigungen durch den Fehler δ_j^l auszudruecken. Zuerst einmal fuer die Neigung gilt:

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^{l+1}} \frac{\partial z_j^{l+1}}{\partial b_k^l}$$

Der erste Term ist hierbei per Definition unser Fehler δ_j^l und der zweite Term laesst sich zu 1 evaluieren, da die Summe z_k^{l+1} nur aus b_k^l und sonstigen Konstanten Termen besteht. Somit ist die Ableitung der Neigung gerade unser Fehler:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

Somit ist der Kostengradient bezuglich der Neigung gerade der Fehlervektor.

$$\boldsymbol{\nabla}_{\boldsymbol{b}^l} C = \boldsymbol{\delta}^l \tag{BP3a}$$

Fuer die partiellen Ableitungen der Kosten nach dem Gewicht giltet:

$$\frac{\partial C}{\partial w_{j,k}^l} = \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial w_{j,k}^l} = \delta_k^{l+1} \cdot a_j^l \tag{BP4}$$

Uns somit ist der Graidient der Kosten respektive einer Gewichtmatrix: UEBERPRUEFEN

$$\boldsymbol{\nabla}_{\boldsymbol{W}^l} C = \boldsymbol{\delta}^{l+1} (\boldsymbol{a}^{\boldsymbol{l}})^{\mathsf{T}} \tag{BP4a}$$

(Nielsen, *Neural Networks and Deep Learning*)

2.6 Universal Approximation Theorem

Neural Network kann alles erlernen

2.7 Vanishing Gradient Problem

Gleichung BP1 sagt uns, dass der Lernprozess langsam ist, falls φ fast 0 oder 1 ist. Sigmoid. Das Neuron ist saturiert und lernt nicht mehr. In letzter Schicht

2.7.1 Kreuzentropie-Kostenfunktion

2.7.2 ReLU

Kapitel 3

Convolutional Neural Networks

Viele Anwendungen von Machine Learning sind verbunden mit Bild- oder Audioverarbeitung, wie z.B Bildklassifizierung, Gesichtserkennung oder Spracherkennung. Vorallem aber fuer hochaufloesende Bilder sind die KNNs, die wir soeben kennengelernt haben, nicht geeignet. Sie sind zum Teil gar nicht in der Lage eine Korrelation zwischen den Inputs und Outputs zu erlernen. Um diesen Umstand zu erklaeren, wird nun ein kleines Beispielmodell erlauetert:

Es soll ein KNN designed werden, welches eine Photographie klassifizieren soll, ob darauf ein Hund sichtbar ist oder nicht. Wir nehmen fuer dieses Gedankenexperiment bereits ein relativ niedrig aufgeloestes Bild mit 256×256 Pixel, dies entspricht weniger als 0.07 Megapixel (ein iPhone XS hat eine Kamera mit 12 Megapixel). Um die verschiedenen Farben zu codieren besitzt jeder Pixel drei Komponten R, G und B. Somit hat dieses Bild insgesamt $256 \times 256 \times 3 = 196'608$ Komponenten. Jede Komponente ist ein Feature welches das KNN zu verrechnen hat. Somit bestuende die erste Schicht des Netzwerkes aus fast $200'000$ Neuronen. Um diese Schicht nun mit seiner Nachbarsschicht, welche gleiche Dimensionen besitzt, zu verbinden, braucht es $196'608 \times 196'608 = 38'654'705'664$ Verbindungen und damit gleich so viele Gewichte! Fuer ein Netwerke ohne eine einzige Zwischenschichten gaebe es also ueber 38 Milliarden Modellparameter zu erlernen! Das dies nicht realistisch ist, sollte auf der Hand liegen.

Nicht nur die Anzahl Modellparameter sind ein Problem fuer KNNs in der Bildverarbeitung, sondern es bestehen noch weiter Probleme. Trotzdem sollte nun klar sein, dass eine andere Modellarchitektur noetig ist, um Machine Learning auf Bilder anzuwenden. Fuer genau solche Anwendungen wurde eine modifizierte Version eines KNNs entwickelt: das **Convolutional Neural Network** (CNN). Im Allgemeinen sind CNNs immer geeignet, um Daten zu verarbeiten, welche eine rasterartige Form haben, wie eben z.B Bilder. Diese Art von Netzwerk, macht Gebrauch von Konzepten aus der klassischen Bildverarbeitung, wie sie mit beispielsweise Photoshop gemacht werden kann. Wie beim Perzeptron und bei den klassischen KNNs auch, ist die Architektur biologisch inspiriert. Der folgende Abschnitt wird die Funktionsweise eines solchen CNNs erklaeren.

(Goodfellow, Bengio und Courville, *Deep Learning*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Convolutional neural network — Wikipedia, The Free Encyclopedia*)

3.1 Bilder als Tensoren

CNNs operieren an Bildern. Sie stellen den Input fuer die Modells dar. Um mit ihnen rechnen zu koennen, ist es sinnvoll sie als sogennante **Tensoren** vom Rang 3 zu untersuchen, anstatt

sie als Anordnungen von Pixeln zu betrachten.
Um zu verstehen, was ein Tensor dritten Ranges ist, muessen wir zuerst verstehen, was ein Tensor im Allgemeinen ist.

Tensor

Ein Tensor $\overline{\mathbf{T}}$ ist eine Verallgemeinerung von Skalaren, Vektoren und Matrizen auf n Dimensionen. Es handelt sich wie bei Matrizen um eine Zahlenanordnung. Dabei wird die Anzahl Dimensionen, innerhalb welchen die Zahlen liegen als Rang oder Stufe n des Tensors bezeichnet. Vorstellen kann man sich einen Tensor als ein Hyperrechteck mit n Dimensionen, innerhalb dessen die Zahlen in einem Raster angeordnet sind. Diese Zahlen sind die Elemente des Tensors. Ein Tensor nullten Ranges ist ein Skalar, einer erster Stufe ein Vektor und einer mit Rang 2 ist eine normale Matrix.

$$1 \in \mathbb{R} \text{ (Skalar)} \quad \begin{pmatrix} 1 & 2 & 3 \end{pmatrix} \in \mathbb{R}^3 \text{ (Vektor)} \quad \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2\times 3} \text{ (Matrix)}$$

Tensor 3. Ranges

Eine Tensor $\overline{\mathbf{T}} \in \mathbb{R}^{h\times w\times d}$ mit Rang 3 ist eine 3-dimensionale Zahlenanordnung. Man kann sich diesen Tensor einfach als eine 3D-Matrix vorstellen. Ein Volumen innerhalb dessen Zahlen in einem Raster angeordet sind. Analog zum Volumen, bezeichnet man die Form des Tensors mit Hoehe, Breite und Tiefe.
Ein Tensor von der Form $\mathbb{R}^{3\times 3\times 3}$ koennte folgendermassen aussehen:

Ein Bild kann somit einfach als Tensor dritten Ranges $\overline{\mathbf{B}} \in \mathbb{R}^{h\times w\times c}$, der Form (Bildhoehe $\times$ Bildbreite $\times$ Anzahl Farbkomponenten) betrachtet werden. Elemente der Matrix nehmen dann einfach die Werte der Pixelkomponenten an. Ein schwarzweiss Bild hat nur eine Komponente, welche die Helligkeit angibt. Somit waere es eine normale 2D-Matrix $\boldsymbol{B} \in \mathbb{R}^{h\times w}$. Siehe Abb. (3.1).

Abbildung 3.1: Beispiel einer Bildmatrix

3.2 Topologie von CNNs

Ein CNN besteht, wie ein KNN auch, aus Schichten. Jede dieser Schichten erhaelt ein Bild als Input und hat auch wieder eines als Output. Somit nehmen die Bilder die Rolle der Aktivierungen a ein, weshalb man sie mit $\overline{\mathbf{A}}^l$ bezeichnet. Das Bild $\overline{\mathbf{A}}^0$ ist hierbei wieder der Input $\overline{\mathbf{X}}$ ins ganze Netz.

Ein wichtiger Unterschied eines CNNs ist, dass es verschiedene Typen von Schichten gibt. Man unterscheiden zwischen drei Arten von Schichten:

- **Fully-connected Schichten**,
- **Convolutional Schichten**, und

- **Pooling Schichten**.

Die fully-connected Schicht ist altbekannt und ist einfach die klassischen Schicht eines KNNs, bestehend aus Neuronen. Die Convolutional Schicht und die Pooling Schicht sind zwei neuartige Schichten, welche eine Erweiterung fuer CNNs darstellen. Sie werden in diesem Kapitel erklaert. Diese beide neuen Arten von Schichten bildet gewissermassen eine Einheit, denn auf eine Convolutional Schicht folgt auch immer eine Pooling Schicht.

In Abbildung (3.2) ist ein Schema eines CNNs abbgebildet.

Abbildung 3.2: Schichtung eines CNNs

3.3 Filter

Zuerst betrachten wir die Convolutional Schicht. Diese Schicht macht ausgiebig Gebrauch von sogennanten Filtern. Diese werden in diesem Abschnitt behandelt.

3.3.1 Filter in der Bildverarbeitung

Filter (auch Kerne) sind in der Bildverarbeitung sehr verbreitet. Jeder kennt sie entweder von Photoshop, von Instagram oder sonstigen Bildbearbeitungsprogrammen. Auch CNNs machen Gebrauch von solchen Filtern, in diesem Fall um die Features eines Bildes zu erlernen. Aus diktaktischen Gruenden betrachten wir zuerst nur 2D-Filter, welche sich fuer graustufige Bilder $\boldsymbol{B} \in \mathbb{R}^{h \times w}$ eignen.

Abbildung 3.3: der Instagramfilter: Blablabla

Ein Filter ist einfach eine Regionen, deutlich kleiner als das verarbeitete Bild, welche ueber alle Pixel wandert, diese manipuliert und so wieder ein neues Bild $\boldsymbol{B}^*$ liefert. Mathematisch gesehen handelt es sich bei einem Filter einfach um einen Tensor, den sogennanten **Filtertensor** oder Faltungstensor (siehe Abb. (3.4)). Fuer unser graustufiges Bild ist dieser eine 2D-Matrix, bezeichnet mit $\boldsymbol{F} \in \mathbb{R}^{f \times f}$. Solche Filter sind immer quadratisch, wobei ihre Zeilen- bzw. Spaltenlaenge mit f bezeichnet wird. Dabei ist f immer eine ungerade Zahl, damit ein Element, das sogennante **Zentralelement**, immer im Zentrum des Filters liegt.

Abbildung 3.4: eine 2D-Filtermatrix

Das Verhalten des Filters wird durch seine Matrixeintraege bestimmt. Die Elemente koennen also so gewaehlt werden, dass wenn der Filter auf ein Bild angewandt wird, er bestimmte Features aus dem Bild hervorhebt. Solche Features koennten zum Beispiel Raender oder Kanten sein, welche betont werden.

Ein gutes Beispiel ist der Sobel-Filter. Er ist ein typischer Kantendetektions-Filter mit einer Groesse von $f = 3$. Wir betrachten nun den Sobelfilter $\boldsymbol{G}_x$, welcher horizontale Kanten akzentuiert. Die Eintraege dieses Filters lauten wie folgt:

$$\boldsymbol{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Angewandt auf ein Bild, erhaelt man ein neues Bild, bei welchem alle erkannten Katen weiss eingefaerbt sind und die restlichen Bereichen schwarz sind. Der Filter erkennt dabei jede Stelle als Kante, welche zwei Regionen mit genug grossem Kontrast voneinander trennt. In Abbildung (3.5) wurde der horizontale Sobelfilter auf ein Beispielbild angewandt. Links ist das Ursprungsbild und rechts ist das prozessierte Bild.

Abbildung 3.5: Sobel-Filter angewandt auf Beispielsbild

(Mehrere, *Sobel operator — Wikipedia, The Free Encyclopedia*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Kernel (image processing) — Wikipedia, The Free Encyclopedia*)

3.3.2 Filter in CNNs

Wie wir vorhin erfahren haben, koennen Filter genutzt werden um gewisse Features eines Bildes hervorzuheben und andere Features zu ignorieren. Dabei bestimmen die Filtereintraege, welche Features extrahiert werden. Somit bestimmt die Aufgabe darin, die richtigen Filterelemente zu bestimmen, damit die gewuenschten Features erkannt werden und auf die gewuenschten Outputs abgebildet werden. Daraus schliesst sich, dass die Filtereintraege die Modellparameter eines CNNs sind. Mithilfe des Gradietenverfahrens soll erneut diese Parameter erlernt werden. Da die Filter die gleiche Funktion haben, wie die Gewichte in einem KNN, bezeichnet man die Filter in einem CNN mit $\overline{\mathbf{W}}$.

$$\boldsymbol{W} = \begin{pmatrix} w_{1,1} & \cdots & w_{1,f} \\ \vdots & \ddots & \vdots \\ w_{f,1} & \cdots & w_{f,f} \end{pmatrix}$$

Die Groesse der Filters bzw. die Groesse der rezeptiven Felder ist ein Hyperparameter.

3.3.3 Filteroperation intutitiv

Wie wendet man nun einen solchen Filter auf ein Bild an? Um das Prozedere einer Filteroperation leichter verstaendlich zu machen, wird sie erneut fuer ein graustufiges Bild $\boldsymbol{A} \in \mathbb{R}^{h \times w}$ erlauetert. Spaeter werden auch noch farbige Bilder erklaert.

Bei einer Filteroperation, wendet man einen Filter $\boldsymbol{W}$ auf eine Bildmatrix $\boldsymbol{A}$ an und erhaelt so ein neues Bild $\boldsymbol{A}^*$. Man schreibt dafuer: $\boldsymbol{A}^* = \boldsymbol{W} * \boldsymbol{A}$.
Um nun diese Operation zu erklaeren, verwenden wir als Beispiel eine 2D-Filtermatrix $\boldsymbol{W} \in \mathbb{R}^{3\times 3}$, bei welcher $f = 3$ gewaehlt wurde. Wie bereits vorhin erwaehnt wandert der Filter ueber das Bild. Dabei befindet sich der Filter immer ueber einer Region des Bilder, welche gleich gross ist, wie der Filter selber (in diesem Fall (3×3)), so dass den Filtermatrixeintraegen immer eindeutig Bildmatrixeintraege entsprechen. Diese Region des Bildes bezeichnet man als das **Rezeptive Feld** des Filters (siehe Abb. (3.6)). Wir schreiben dafuer $\tilde{\boldsymbol{A}}^{(y,x)}$, wobei die hochgestellten Indizes (y, x) die Position des Zentralelements im rezeptiven Feldes angiebt. Somit ist das rezeptiven Feld einfach eine Untermatrix $\tilde{\boldsymbol{A}} \in \mathbb{R}^{3\times 3}$ des Obermatrix $\boldsymbol{A}$.

Abbildung 3.6: ein Filter und sein rezeptives Feld

Der Filter beginnt nun oben links ueber das Bild zu wandern. Somit hat der Filter zu beginn das rezeptive Feld $\tilde{\boldsymbol{A}}^{(2,2)}$, denn der Filter darf nicht ueber die Raender des Bildes hinausragen. Nun werden die Elemente der Filters mit den Elementen des rezeptiven Feldes verrechnet, welche die gleichen Dimensionen besitzen. Dabei wird einfach das Hadamard-Produkt (das elementweise Produkt) des beiden Matrizen miteinander gebildet. Nun erhalten wir eine neue Matrix $\boldsymbol{W} \odot \tilde{\boldsymbol{A}}^{(2,2)} = \boldsymbol{P} \in \mathbb{R}^{3\times 3}$. Nun werden noch alle Elemente der neuen Matrix $\boldsymbol{P}$ aufsummiert. Diese Zahl ist dann der Grauwert des ersten Pixels $(\boldsymbol{A}^*)_{1,1}$ des bearbeiteten Bildes. In diesem Sinne stellt der Filter eine Gewichtung der Nachbarspixel dar, welche bestimmt, wie die neuen Pixel aussehen.

$$(\boldsymbol{A}^*)_{1,1} = \sum_{y=1}^{3}\sum_{x=1}^{3}(\boldsymbol{P})_{y,x}$$

Nun wird der Filter um ein Element nach rechts verschoben und die gleiche Prozedur angewendet, um den zweiten Pixel zu berechnen. Dies wird durchgezogen, bis eine ganze Zeile der Bildmatrix durchstreift wurde. Danach wird der Filter wieder nach ganz rechts verschoben und er bewegt sich ein Element nach unten. Das geschieht, bis das ganze Bild verrechnet wurde. Dabei werden die Elemente des urspruenglichen Bildes durchaus mehrmals verrechnet, da es zu einer Ueberlappung des vorherigen Filter und des verschobenen Filters kommt.

Abbildung 3.7: Schema, wie ein Filter ueber ein Bild laueft

Es ist zu bemerken, dass das neue Bild $\boldsymbol{A}^* \in \mathbb{R}^{(h-2)\times(w-2)}$ nicht mehr die gleichen Masse hat, wie das Ursprungsbild $\boldsymbol{A} \in \mathbb{R}^{h\times w}$. Das liegt daran, dass pro Lage des Filters jeweils ein Pixel des neuen Bildes berechnet wird. Man kann sich vorstellen, dass an der Position des Zentralelements jeweils ein neuer Pixel entsteht. Dabei muss der Filter immer eine vollstaendige Region als rezeptives Feld haben. Dadurch kommt das Zentralelement nie auf den Raendern des Bilds zu liegen. Somit fallen diese weg.

(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Convolution— Wikipedia, The Free Encyclopedia*)

3.3.4 Filteroperationen als diskrete Faltungen

Eigentlich handelt es sich bei einer Anwendung eines Filters auf ein Bild um eine spezifische mathematische Operation: eine **diskrete Faltung** (engl.: **Convolution**). [1] Daher ruehrt auch der Name des Convolutional Neural Networks. Wir werden die Faltung als mathematische Operation nicht im vollen Umfang betrachten, da sie relativ kompliziert ist. Wir werden uns lediglich auf die Bedeutung der Faltung fuer Filter beschraenken. Damit ihr die Defintion trotzdem mal gesehen habt:

$$(f * g)(x) = \int f(t)g(x-t)\mathrm{d}a = \sum_{t=-\infty}^{\infty} f(a)g(x-t) \tag{3.1}$$

Streng genommen wird in CNN nicht eine diskrete Faltung durchgefuehrt, sondern eine sogennante Kreuzkorrelation. Der Unterschied dabei ist, dass bei einer Kreuzkorrelation die Fal-

[1] Eigentlich ist die Filteroperation keine richtige Faltung im mathematischen Sinne, sondern eher eine sogennante Kreuzkorrelation. Dennoch wird im Verlauf dieser Arbeit der Begriff Faltung im Zusammenhang mit Filteroperationen verwendet.
Fuer Intressierte: Die Kreuzkorrelation funktioniert gleich wie eine Faltung, jedoch wird die eine Matrix zuerst horizontal, wie auch vertikal gespiegelt, bevor die eigentliche Faltung angewandt wird.

tungsmatrix nicht zuerst horizontal, wie auch vertiakl gespiegelt wird. Begrifflich benutzt man jedoch Faltung/Convolution um beide Operationen zu bezeichnen.

Mithilfe der Faltungoperation koennen wir die Schritte welche in Sektion (**??**) erklaert wurden zusammenfassen. Um die Schreibweise etwas zu vereinfachen, fuehren wir folgende Konvention ein:

$$\boldsymbol{M}(x, y) = (\boldsymbol{M})_{y,x}$$

Um einen Pixel $\boldsymbol{A}^*(x, y)$ des neuen Bildes zu berechnen, benutzt man folgende Formel.
Kreuzkorrelation

$$\boldsymbol{A}^*(x, y) = (\boldsymbol{W} * \boldsymbol{A})(x, y) = \sum_{u=0}^{f} \sum_{v=0}^{f} \boldsymbol{A}(x + u, y + v)\boldsymbol{W}(u, v) \tag{3.2}$$

Fuer farbige Bilder ist das Vorgehen praktisch das selbe. Es werden nun lediglich 3D-Tensoren anstatt 2D-Tensoren verwendet. Wir betrachten den allgemeinen Bildtensor $\overline{\mathbf{A}}_{h \times w \times c}$, wobei c die Anzahl Farbkomponenten (engl.: channels) ist. Nun benutzt man Filter $\overline{\mathbf{W}}_{f \times f \times c}$ mit beliebiger Groesse f aber mit der gleicher Tiefe c wie das Ursprungsbild $\boldsymbol{A}$. Das heisst der Filter muss nicht entlang der Tiefe des Bildes wandern, da er gerade gleich tief ist, wie das Bild.
Die allgemine Filtergleichung lautet:

$$\overline{\mathbf{A}}^*(x, y, z) = (\overline{\mathbf{W}} * \overline{\mathbf{A}})(x, y, z) = \sum_{u=0}^{f} \sum_{v=0}^{f} \sum_{w=0}^{c} \boldsymbol{A}(x + u, y + v, w)\boldsymbol{W}(u, v, w) \tag{3.3}$$

3.3.5 Padding

Wie bereits erwaehnt schrumpfen die Bilder (an den Raendern), wenn man ein Filter auf sie anwendet. Zur Erinnerung: Dies liegt daran, dass pro Lage des Zentralelement nur ein Pixel des neuen Bildes entsteht und das Zentralelement nunmal nicht auf allen Ursprungspixeln zu liegen kommt. Die Raender fallen so weg. Diese Groesse des Wegfalls haengt von der Filtergroesse f ab. Hierbei giltet folgende Formel fuer die neue Groesse n_1 des Bildes, anhand der alten Groesse n_0.

$$n_1 = n_0 - f + 1 \tag{3.4}$$

Das Problem hierbei ist, dass nach einigen Faltungen das Bild extrem geschrumpft ist und im Grenzfall kleiner als der Filter wird. Das darf natuerlich nicht passieren. Ausserdem geht dabei Information an den Raender verloren, da es im Innern des Bildes zu viel Overlapping der Filterlagen kommt und die Raeder nur sehr wenig einbezogen werden.

Diese Probleme werden mit sogennantem **Padding** behoben. Padding ist ein Schritt, welcher vor der eigentlichen Faltung stattfindet. Dabei werden einfach zusatzliche Raender (Zeilen und Spalten) an das Ursprungsbild angebracht. Die Matrix wird an den Enden erweitert. Die neuen Elemente werden dabei auf den Wert 0 gesetzt.

Abbildung 3.8: Padding

Das Padding p ist eine Zahl, welche angibt wieviele Elemente an allen Raendern hinzugefuegt werden. Padding $p = 1$ bedeutet, dass an allen Kanten jeweils eine Reihe bzw. Spalte hinzugefuegt wird. Begrifflich unterscheidet man zwei Arten von Padding:

- valid-Padding: Es werden keine zuesaezlichen Elemente angebracht. f ist also 0.
- same-Padding: Es werden so viele Reihen und Spalten angebracht, dass sich die Groesse des Bildes bei der Faltung erhalten bleibt.

Um same-padding zu erhalten, muss man p so waehlen, dass es den Wegfall durch den Filter gerade kompensiert. Da durch das Padding der Filter jeweils um zwei p groesser wird, erhalt man $n_1 = n_0 - f + 1 + 2p$. Wenn man diese Formel jetzt nach p aufloest, erhaelt man folgende Formel fuer das Waehlen des valid-Paddings p.

$$p = \frac{f-1}{2} \tag{3.5}$$

3.3.6 Stride

Bis jetzt haben wir bei den Filterfaltungen, den Filter pro Anwendung immer nur um einen Pixel verschoben. Das muss aber nicht so sein. Man kann den Filter auch mit einer anderen Schrittgroesse verschieben. Diese Eigenschaft bezeichnet man als **Stride** s. Falls $s = 2$ gewaehlt wird, bedeutet das, dass der Filter sich pro Matrixmultiplikation um zwei Elemente bewegt. Somit wurde eine Position uebersprungen. Zur Folge hat dies, dass das neue Bild $\boldsymbol{B}^*$ deutlich kleiner geworden ist, denn das Zentralelement uebspringt somit auch diese Felder und bildet so deutlich weniger Pixel.

Folgende Formel beschreibt die Dimensionen des neuen Bildes $\boldsymbol{B}^*$ unter beruecksichtigung der Filtergroesse f, dem Padding p und dem Stride s.

$$n_1 = \frac{n_0 + 2p - f}{s} + 1 \tag{3.6}$$

3.3.7 Mehrere Filter

Eine Filteroperation ist nicht auf einen einzigen Filter beschraenkt. Es koennen mehrere Filter auf das gleiche Ausgangsbild angewandt werden und zusammen ein Endbild erzeugen.

Die Anzahl Filter bezeichnen wir mit c. Nun macht man pro Filter eine Faltung ueber das Ursprungsbild $\overline{\mathbf{B}}$, wobei jede Faltung eine neue Matrix $\boldsymbol{B}_i^*$ liefert, wobei i der Index des Filters ist. All diese Tensoren sind zweidimensional, unabhaengig davon, wieviel Komponten das Ursprungsbild hatte. Aus diesem Grund koennen die einzelnen Matrizen $\boldsymbol{B}_i^*$ aufeinander gelegt werden und somit einen grossen 3D-Endtensor $\overline{\mathbf{B}}^*$ bilden. Hierfuer muessen alle Filter gleich gross sein. Die Tiefe des neuen Bildes $\overline{\mathbf{B}}^*$ ist jetzt gerade die Anzahl Filter c. Wir haben vorhin bereits immer die Tiefe des Bildes (bzw. die Anzahl Farbkomponenten) mit c bezeichnet.

3.3.8 Wieso Filter?

Es stellt sich nun die Frage, weshalb sich Filter fuer Machine Learning eignen. Wie bereits erwaehnt besteht die Aufgabe von Filtern darin bestimmte Features eines Bildes hervorzuheben und die restlichen auszublenden. Genau die gleiche Aufgabe erfuellen die Neuronen in einem KNNs. Auch sie sollen Features der Inputsdaten erlernen, wobei gewisse Neuronen auf gewisse Features reagieren.

Jedoch ist zu erkennen, dass Bilddaten sich deutlich von sonstigen Daten unterscheiden. Bildfeatures bzw. Pixel sind immer im Kontext von ihren Nachbarn wichtig. Denn ein Pixel ist erst eine Kante oder eine Ecke, wenn er zwei verschiedene Kontraste voneinander trennt. Oder um die Konturen eines Gegenstandes zu erkennen, muss man eine ganze Fuelle von Pixeln betrachten. Man bezeichnet dies als **lokalisierte Features**.

Ein weiterer Aspekt besteht darin, dass Bildausschnitt nicht immer gleich orientiert sind. Wenn man ein Gesicht auf einem Bild erkennen moechte, sollte es keine Rolle spielen, wo auf dem Bild es sich befindet, oder welche Groesse es hat. Es besteht der Wunsch nach **Invarianz**.

Typische Invarianzen, welcher ein Filter erfuellt, sind: Translationinvarianz, Rotationsinvarianz und Helligkeitsinvarianz.
Parameter Sharing
Variable Inputgroesse, verschieden grosse Bilder
Sparse connectivity: Nicht alle Werte werden miteinander verrrechnet deutlich schneller als Matrixmultiplikation.

3.4 Convolutional Layer

Da wir nun die Filter in ihrem vollen Umfang betrachtet haben, koennen wir uns endlich den Convolutional Schichten widmen. Eine ConvSchicht l erhaelt als Input ein Bild $\overline{\mathbf{A}}^{l-1} \in \mathbb{R}^{h^{l-1} \times w^{l-1} \times c^{l-1}}$. Darauf werden als erstes alle c^l Filter $F_i^l \in \mathbb{R}^{f^l \times f^l \times c^{l-1}}$ angewandt. Die neuen Matrizen A_i^* werden aufeinander draufgelegt und bilden den Tensor $\overline{\mathbf{A}}^{*,l} \in \mathbb{R}^{h^l \times w^l \times c^l}$. Nun kommt ein Optionaler Zwischenschritt. Man kann nun auf jedes Element des Ausgangstensor einen Bias b draufaddieren.
Nun wendet man auf alle Elemente des Ausganstensor die vektorisierte Aktiverungsfunktion φ an. Die gaengiste Aktiverungsfunktion ist hierfuer die ReLU-Aktiverungsfunktion φ^{ReLU}. Dadurch erhaelt man den Aktiverungstensor $\overline{\mathbf{A}}^l \in \mathbb{R}^{h^l \times w^l \times c^l}$.
Da die Faltungsoperation eine lineare Operation, braucht man wieder eine Aktivierungsfunktion.
Eigentlich ist ein CNN einfach ein KNN, welches in einer Schicht anstatt Matrixmultiplikation eine Faltungsooperation benutzt.
(Goodfellow, Bengio und Courville, *Deep Learning*)

3.5 Pooling Layer

Pooling reduziert die Dimensionalitaet der Daten, indem eine Gruppe Tensorelementen der vorherigen Schicht zu einem einzigen Tensorelement der naechsten Schicht zusammengefasst wird.
Viel Begrifflichkeit der Filter wird auf das Pooling uebertragen. Die Groesse des Felder, welches zusammengefasst wird, bezeichnet man mit f. Das Stride s bezeichnet auch hier wieder, was die Schrittgroesse bei Verschieben des Feldes ist. Local Pooling 2x2 Feld wird zu einem Neuron zusammengefasst. MaxPooling -> behaelt einfach maximalen Wert des Feld bei und verwirft den Rest. Average Pooling berechnet das arithmethische Mittel des Feldes und verwednet dieses. MaxPooling ist am effektivsten.
Pooling: Komprimierung des Bildes. Durchschnitt von 2x2 Pixeln berchnen -> ein Pixel des gepoolten Bild. -> 75% Informationsreduktion .
Pooling als Filter betrachtet: MaxPooling mit 2x2 Feld. Filtergroesse $f = 2$ und stride $s = 2$. Dies sind die Hyperparamter des MaxPooling. Hat also zwar Hyperparameter aber keinerlei Modellparameter
MaxPooling koennte auch andere Filtergroesse oder Stride haben. Formeln fuer Outputgroesse gelten auch hier.
Bei 3D-Tensoren aendert sich die Depth nicht, da das Pooling auf jeden Channel einzeln angewandt wird.
Es gibt zwei Konventionen: Entweder bezeichnet man die Einheit von einem ConvLayer und einem PoolLayer zu einer Schicht zusammengefasst werden oder sie werden als seperate Schichten betrachtet...

3.6 Fully-connected Layer

Flattening ist, wenn man einen Tensor zu einem Vektor umformt

Kapitel 4

Autoencoder

Ein **Autoencoder** ist eine weitere Architektur eines KNNs. Er wird benutzt um effiziente Codierungen von Daten unsupervised zu erlernen.
Der Autoencoder soll eine komprimierte Repraesentation (Encoding) fuer einen Datensatz erlernen und somit die wesentlichen Features herausextrahieren. Er kann so zur **Dimensionalitaetsreduktion** verwendet werden. Dabei faellt das "Rauschen"(engl.:Noise) der Daten weg. Das Netzwerk rekonstruiert dann anhand der Codierung wieder die urspruenglichen Daten.

4.1 Topologie von Autoencodern

- Eingabeschicht
- Flaschenhals/Encoding
- Ausgabeschicht/Decoding

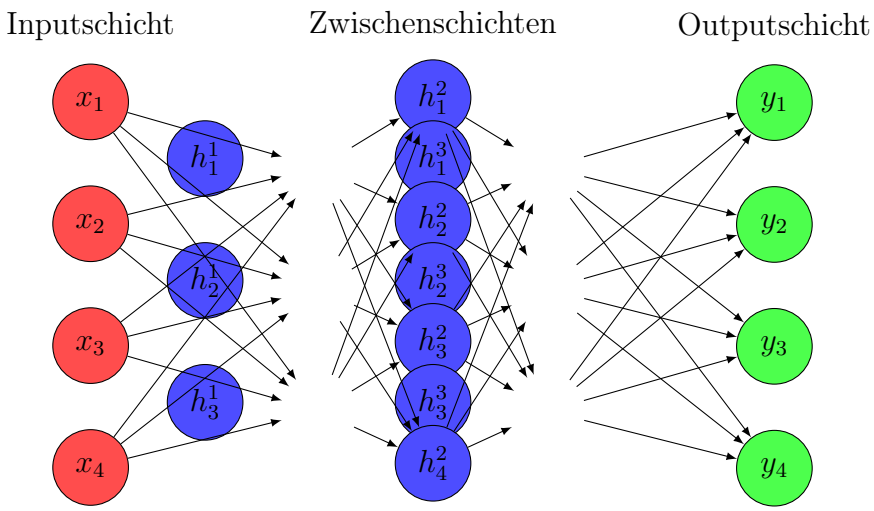


Abbildung 4.1: Schichten eines Autoencoders

4.2 Mathematische Betrachtung

Falls lineare Neuronen verwendet werden, ist der Prozess der Dimensionalitaetsreduktion sehr aehnlich zur Hauptkomponentenanalyse.

Mathematik:

$$\phi : \mathcal{X} \to \mathcal{F} \tag{4.1}$$
$$\psi : \mathcal{F} \to \mathcal{X} \tag{4.2}$$
$$\phi, \psi = \arg\min_{\phi,\psi} \|X - (\phi \circ \psi)X\|^2 \tag{4.3}$$

4.3 Bedeutung von Autoencodern

4.3.1 Datenkompression

Falls ein Autoencoder nur aus linearen Neuronen besteht, ist die Dimensionsreduktion sehr aehnlich zu der Hauptkomponentenanalyse.
Problem: Netzwerk versucht Durchschnitt aller Trainingsdaten zu erlernen.

4.3.2 Datengenerierung

4.3.3 Eigenschaften erkennen

4.4 Variational Autoencoder?

4.5 Detangled Autoencoder?

4.6 Autoencoder als generative Modelle

Erwaehnung von GANs

4.7 Programmiertools

4.7.1 Tensorflow

Kapitel 5

Entwicklung eines Autoencoders

5.1 Planung/Methoden

5.1.1 Idee

Gesichtssynthese mithilfe eines Autoencoders als generatiaves Modell.

5.1.2 Methoden/Tools

5.1.2.1 Python

5.1.2.2 Tensorflow

5.1.2.3 Keras

5.1.2.4 Sonstige "Tools"

- Spacemacs (Emacs)
- Arch Linux

5.1.3 Trainingsdaten

MegaFace Dataset

5.2 Daten-Normalisierung

Die Gesichter muessen "normalisiert" werden. In diesem Fall bedeutet, dies dass alle Gesichter gleiche Ausrichtung und Skalierung haben muessen. Dazu muessen sie verschoben, rotiert und skaliert werden.

5.2.1 Daten laden

5.2.2 Facealigner

5.2.2.1 Facial landmarks

Shape prediction Problem. Erkennung von lokalen Gesichtsfeatuers und bestimmung ihrer Position im Bild.
Zwei Schritte: Gesicht lokalisieren und erkennung der Gesichtstrukturen im Bild

DLib implementation: Beispiel der Implemenation von One Millisecond Face Alignment with an Ensemble of Regression Trees paper by Kazemi and Sullivan (2014).
(Rosebrock, *Facial landmarks with dlib, OpenCV, and Python*)

5.2.2.2 der Aligner

Hierfuer werden wir einen "Facealigner" in Python schreiben, mithilfe von OpenCV und dlib (ueber AUR installiert).
(Rosebrock, *Face Alignment wirth OpenCV and Python*)

5.2.3 Zuschneiden

5.3 Programmieren

Anhang A

Beispielsanhang

Literatur

[1] Andrew NG - deeplearning.ai. *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*. [Online; besucht am 17. Juni 2019]. 2017. URL: https://www.youtube.com/playlist?list=PLkDaE6sCZn6Gl29AoE31iwdVwSG-KnDzF.

[2] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[3] Mehrere. *Convolution— Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Convolution.

[4] Mehrere. *Convolutional neural network — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Convolutional_neural_network.

[5] Mehrere. *Kernel (image processing) — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Kernel_(image_processing).

[6] Mehrere. *Normal distribution — Wikipedia, The Free Encyclopedia*. [Online; besucht am 26. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Normal_distribution.

[7] Mehrere. *Perzeptron — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 28. April 2019]. 2019. URL: https://de.wikipedia.org/wiki/Perzeptron.

[8] Mehrere. *Sobel operator — Wikipedia, The Free Encyclopedia*. [Online; besucht am 17. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Sobel_operator.

[9] Mehrere. *Supervised learning — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Supervised_learning.

[10] Mehrere. *Unsupervised learning — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Unsupervised_learning.

[11] Michael Nielsen. *Neural Networks and Deep Learning*. http://neuralnetworksanddeeplearning.com. 2018.

[12] Adrian Rosebrock. *Face Alignment wirth OpenCV and Python*. [Online; besucht am 4. Juli 2019]. 2017. URL: https://www.pyimagesearch.com/2017/05/22/face-alignment-with-opencv-and-python/.

[13] Adrian Rosebrock. *Facial landmarks with dlib, OpenCV, and Python*. [Online; besucht am 4. Juli 2019]. 2017. URL: https://www.pyimagesearch.com/2017/04/03/facial-landmarks-dlib-opencv-python/.

Abbildungsverzeichnis

1.1 Schema eines Modells . . . 3
1.2 beschriebenes Beispielmodell . . . 3
1.3 Skalarfeld mit zugehoerigem Gradientenfeld . . . 5
1.4 Visualisierung des Gradientenabstiegs . . . 6
1.5 Visualisierung verschiedener Lernraten . . . 6

2.1 Perzeptron mit drei Inputs . . . 9
2.2 erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts) . . . 9
2.3 Nervenzelle . . . 9
2.4 ein kuenstliches Neuron und seine Bestandteile . . . 10
2.5 Definition und Graph der Heaviside-Funktion Θ . . . 10
2.6 Formel und Graph einer stueckweisen linearen Funktion . . . 11
2.7 Definition, Ableitung und Graph der Sigmoid-Funktion σ . . . 11
2.8 Schichten eines KNNs . . . 12
2.9 zum Verstaentniss der Nomenklatur . . . 13
2.10 Neuron mit Gewichtbeschriftungen und Neigung . . . 13
2.11 Graph der Dichtefunktion $\phi(x \mid \mu = 0, \sigma^2 = 1)$ mit $X \sim \mathcal{N}(0, 1)$ . . . 16
2.12 Computation Graph einer exemplarischen Verkettung von Funktionen . . . 17
2.13 Computation Graph zur Berechnung von δ_j^L . . . 17
2.14 Computation Graph zur Berechnung von δ_j^{l-1} . . . 19

3.1 Beispiel einer Bildmatrix . . . 22
3.2 Schichtung eines CNNs . . . 23
3.3 der Instagramfilter: Blablabla . . . 23
3.4 eine 2D-Filtermatrix . . . 23
3.5 Sobel-Filter angewandt auf Beispielsbild . . . 24
3.6 ein Filter und sein rezeptives Feld . . . 24
3.7 Schema, wie ein Filter ueber ein Bild laueft . . . 25
3.8 Padding . . . 26

4.1 Schichten eines Autoencoders . . . 30

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth